FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

BASE DE DATOS AVANZADAS Y BIG DATA

Solución de Inteligencia de Negocios y Solución Big Data

Diseño e implementación de una solución BI basada en ETL y Data Warehouse para el análisis de ventas de un restaurante

DATOS DEL EQUIPO	INFORMACIÓN	
Autores	Arroyo Luna, David Orestes	100%
	Chavarry Miranda, Brenda Fiorella	100%
	Lucas Marmolejo, Erick Giordano	100%
	Taipe Figueroa, Jhon Victor	100%
Docente	HUGO ALEJANDRO MAMANCHURA LIMA	
Curso / NRC	Base de Datos Avanzadas y Big Data / 7262	
Repositorio		
Período	2026-2	

Lima - Perú

2026

ORIENTACIONES PARA EL PROYECTO Y EL INFORME

Propósito del proyecto: desarrollar, integrar y evaluar una solución de Inteligencia de Negocios y una solución Big Data que respondan a una necesidad empresarial verificable. El informe debe demostrar decisiones de ingeniería, experimentación reproducible y dominio técnico de ambas soluciones.

Alineación referencial con ABET: este formato operacionaliza el Student Outcome 6 de ABET EAC mediante evidencias directas de diseño y ejecución de experimentos, análisis e interpretación de datos y conclusiones sustentadas con juicio de ingeniería.

Entregables obligatorios

N.°	ENTREGABLE	EVIDENCIA MÍNIMA
1	Informe técnico	Documento completo con citas válidas, tablas, figuras y trazabilidad de evidencias.
2	Solución BI functional	Modelo dimensional, ETL/ELT, almacén de datos o data mart, indicadores, análisis OLAP y dashboard.
3	Solución Big Data functional	Ingesta, almacenamiento y procesamiento escalable o distribuido, consultas/análisis y demostración técnica.
4	Paquete experimental	Dataset original y procesado, scripts/notebook, configuraciones, logs, métricas, resultados y visualizaciones.
5	Repositorio versionado	Código, documentación, historial de cambios, etiqueta de versión final y evidencia de aportes individuales.
6	Defensa y mejora	Demostración funcional, respuesta individual, retroalimentación recibida y mejoras implementadas.
7	Uso responsable de IA	Declaración de herramientas, propósito, verificación, autoría intelectual, privacidad y limitaciones.

Convenciones de presentación de evidencias

1. Identifique cada evidencia con un código único (por ejemplo, E-01, E-02) y úselo en el informe, anexos y repositorio.
2. Todo resultado debe indicar dataset, versión del código, configuración, fecha de ejecución y responsable.
3. Los enlaces deben ser accesibles para el docente y conservar una versión congelada del entregable final.
4. No reemplace el análisis técnico por capturas: las capturas complementan, pero no sustituyen, datasets, scripts, métricas, tablas y logs.
5. La evaluación del resultado de experimentación requiere evidencia individual además del producto grupal.

ÍNDICE

Actualice el índice automático después de completar el informe. Incluya además los índices de tablas, figuras y anexos.

CAPÍTULO I: CONTEXTO, PROBLEMA Y OBJETIVOS 5

1.1 Contexto y necesidad empresarial 5

1.2 Formulación del problema de ingeniería 5

1.3 Objetivos 5

1.4 Alcance, restricciones y supuestos 6

1.5 Stakeholders y criterios de éxito 6

1.6 Fuentes, gobierno y ética de los datos 7

CAPÍTULO II: ANTECEDENTES Y FUNDAMENTOS 8

2.1 Antecedentes 8

2.2 Fundamentos de Inteligencia de Negocios 8

2.3 Fundamentos de Big Data 8

2.4 Tecnologías, estándares y trabajos relacionados 8

CAPÍTULO III: DISEÑO DE INGENIERÍA DE LAS SOLUCIONES 9

3.1 Requerimientos funcionales y no funcionales 9

3.2 Alternativas y decisiones de arquitectura 9

3.3 Arquitectura integrada y flujo de datos 9

3.4 Diseño de la solución de Inteligencia de Negocios 9

3.4.1 Requisitos, usuarios e indicadores 9

3.4.2 Modelo dimensional y almacén de datos 9

3.4.3 Procesos ETL/ELT y calidad 9

3.4.4 OLAP, reportes y dashboards 9

3.5 Diseño de la solución Big Data 10

3.5.1 Justificación y requisitos Big Data 10

3.5.2 Arquitectura y componentes 10

3.5.3 Modelo y ciclo de vida de datos 10

3.5.4 Escalabilidad, resiliencia y operación 10

3.5.5 Resultados 10

3.6 Seguridad, privacidad y riesgos 10

3.7 Plan de despliegue y aceptación 10

CAPÍTULO IV: DISEÑO DEL EXPERIMENTO 11

4.1 Pregunta experimental, objetivo e hipótesis 11

4.2 Escenario y datasets 11

4.3 Variables, métricas y criterios de aceptación 11

4.4 Entorno, herramientas y reproducibilidad 11

4.5 Matriz de escenarios experimentales 12

4.6 Procedimiento reproducible 12

4.7 Riesgos, validez y limitaciones del experimento 12

CAPÍTULO V: IMPLEMENTACIÓN Y CONTROL DE EVIDENCIAS 13

5.1 Estructura y versión del repositorio 13
5.2 Implementación de la solución BI 13
5.3 Implementación de la solución Big Data 13
5.4 Calidad, seguridad y pruebas 13
5.5 Registro de retroalimentación y mejoras 13
5.6 Contribuciones individuales 14
5.7 Declaración de uso responsable de inteligencia artificial 14
CAPÍTULO VI: EJECUCIÓN, RESULTADOS Y ANÁLISIS DE DATOS 15
6.1 Validación del dataset original y procesado 15
6.2 Registro de ejecuciones 15
6.3 Tabla consolidada de resultados 15
6.4 Visualizaciones y dashboards 16
6.5 Análisis e interpretación técnica 16
6.6 Verificación de funcionalidad y completitud 16
CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES 17
7.1 Matriz objetivo-resultado-conclusión-recomendación 17
7.2 Conclusiones técnicas 17
7.3 Recomendaciones 17
CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL 18
8.1 Tratamiento ético, legal y seguro de los datos 18
8.2 Impactos de las soluciones 18
8.3 Compromiso profesional 18
REFERENCIAS BIBLIOGRÁFICAS 19
ANEXOS Y MANIFIESTO DE EVIDENCIAS 20
CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES 22
7.1 Matriz objetivo-resultado-conclusión-recomendación 22
7.2 Conclusiones técnicas 23
7.3 Recomendaciones 24
CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL 25
8.1 Tratamiento ético, legal y seguro de los datos 25
8.2 Impactos de las soluciones 25
8.3 Compromiso profesional 26

RESUMEN

El presente proyecto tiene como objetivo desarrollar una solución de Inteligencia de Negocios para analizar las ventas y pagos de un restaurante. El problema identificado es que la información se encuentra distribuida en diferentes tablas de la base de datos, lo que dificulta obtener resultados completos y realizar análisis que ayuden a tomar decisiones.

Para solucionar este problema, se utilizó la base de datos BD_PUNTOVENTA, que contiene información de pedidos, productos, documentos de venta y formas de pago. Mediante un proceso ETL se extrajeron, transformaron y cargaron los datos en un Data Warehouse compuesto por seis dimensiones y dos tablas de hechos. Posteriormente, se utilizó Power BI para elaborar indicadores y gráficos que permiten visualizar las ventas por fecha, producto, mozo, caja y forma de pago.

Para las pruebas se utilizó un conjunto reducido de datos, obteniendo 9 registros de ventas y 6 de pagos. El dashboard permite visualizar ventas totales de 150, una cantidad vendida de 12 productos, pagos por 174 y propinas por 6. También se identificaron diferencias entre los montos de ventas y pagos debido a inconsistencias en algunos datos de prueba.

Como complemento, se plantea una etapa experimental de Big Data para evaluar el procesamiento de un mayor volumen de registros y comparar los tiempos de ejecución.

Finalmente, el proyecto demuestra que organizar los datos mediante ETL y un Data Warehouse facilita su análisis y presentación en Power BI, mientras que la experimentación permitirá evaluar la capacidad de crecimiento de la solución.

Palabras clave: Inteligencia de Negocios, Big Data, ETL, Data Warehouse, Power BI.

ABSTRACT

This project aims to develop a Business Intelligence solution to analyze restaurant sales and payments. The main problem identified is that the information is distributed across

different database tables, making it difficult to obtain complete results and perform analyses that support decision-making.

To address this problem, the BD_PUNTOVENTA database was used, containing information about orders, products, sales documents, and payment methods. Through an ETL process, data was extracted, transformed, and loaded into a Data Warehouse consisting of six dimensions and two fact tables. Power BI was then used to create indicators and charts showing sales by date, product, waiter, cash register, and payment method.

A small test dataset was used, resulting in 9 sales records and 6 payment records. The dashboard displayed total sales of 150, 12 products sold, payments amounting to 174, and tips totaling 6. Differences between sales and payment amounts were also identified due to inconsistencies in some test records.

As an additional component, an experimental Big Data stage is proposed to evaluate the processing of larger datasets and compare execution times.

Finally, the project shows that organizing data through ETL and a Data Warehouse makes it easier to analyze and visualize information in Power BI, while the experimental stage will help evaluate the scalability of the solution.

Keywords: Business Intelligence, Big Data, ETL, Data Warehouse, Power BI.

CAPÍTULO I: CONTEXTO, PROBLEMA Y OBJETIVOS

1.1 Contexto y necesidad empresarial

Los restaurantes generan diariamente información relacionada con pedidos, productos, ventas, clientes y formas de pago. Estos datos son importantes porque permiten conocer el comportamiento de las ventas y pueden servir como apoyo para tomar decisiones sobre la operación del negocio.

Para este proyecto se utiliza la base de datos BD_PUNTOVENTA, donde la información se encuentra distribuida en las tablas MPEDIDO, DPEDIDO, TPRODUCTO, MDOCUMENTO, DDOCUMENTO y DPAGODOCUMENTO. Estas tablas almacenan los pedidos, productos, documentos de venta y pagos realizados.

Actualmente, para obtener información completa de una venta es necesario relacionar varias tablas. Además, una venta puede incluir diferentes productos y utilizar más de una forma de pago, por lo que un análisis directo de los datos podría generar duplicidad o resultados incorrectos. Por este motivo, en el proyecto se separaron las ventas y los pagos en dos tablas de hechos.

Ante esta necesidad, se propone una solución de Inteligencia de Negocios que organice la información en un Data Warehouse y permite analizarla mediante Power BI. De esta manera, se busca conocer las ventas totales, productos más vendidos, ventas por fecha, mozo y caja, así como las formas de pago más utilizadas.

Como línea base, las pruebas realizadas generaron 9 registros de ventas y 6 registros de pagos en el Data Warehouse. Estos datos permitieron comprobar el funcionamiento inicial del proceso ETL y del dashboard. A partir de esta solución también se plantea evaluar, de manera experimental, el procesamiento de un mayor volumen de información.

1.2 Formulación del problema de ingeniería

El principal problema es que la información de ventas y pagos del restaurante se encuentra distribuida en diferentes tablas de la base de datos operacional, lo que dificulta obtener información integrada para realizar análisis y apoyar la toma de decisiones.

Además, las ventas y los pagos manejan diferentes niveles de detalle. Un documento puede incluir varios productos y utilizar más de una forma de pago, por lo que relacionar directamente estos datos podría generar registros duplicados. Para evitar este problema, el modelo desarrollado utiliza dos tablas de hechos independientes: DW_FACT_VENTAS y DW_FACT_PAGOS.

Como solución, se plantea utilizar Inteligencia de Negocios para extraer, transformar y organizar los datos en un Data Warehouse, permitiendo posteriormente analizar la información mediante Power BI.

Actualmente, el proyecto trabaja con un conjunto reducido de datos de prueba. Por ello, el componente de Big Data se plantea como una etapa experimental, en la que se evaluará el procesamiento de una mayor cantidad de registros y la posibilidad de incorporar nuevas fuentes de información. De esta manera, se podrá analizar la escalabilidad de la solución ante un futuro crecimiento de los datos.

1.3 Objetivos

Objetivo general: Desarrollar y evaluar una solución de análisis de datos para las ventas y pagos de un restaurante, integrando una arquitectura de Inteligencia de Negocios basada en ETL, Data Warehouse y Power BI, y complementandolo con una experimentación orientada a evaluar el procesamiento de datos a mayor escala.

Objetivos específicos:

- Identificar e integrar las tablas de la base de datos operacional que contienen la información necesaria para analizar las ventas y pagos del restaurante.
- Diseñar un modelo dimensional que permita analizar la información por fecha, producto, cliente, mozo, caja y forma de pago.
- Implementar un proceso ETL para extraer, transformar y cargar la información desde BD_PUNTOVENTA hacia el Data Warehouse.
- Construir indicadores y visualizaciones en Power BI para analizar ventas, cantidad de productos vendidos, pagos, propinas, productos de mayor movimiento, ventas por fecha, mozo y caja.
- Validar la consistencia de la información cargada en las dimensiones y tablas de hechos.
- Diseñar una prueba experimental con un conjunto de datos de mayor tamaño que permita evaluar el comportamiento y escalabilidad del procesamiento.
- Comparar los resultados obtenidos con la solución tradicional y la alternativa de procesamiento de mayor escala considerando tiempo de procesamiento, cantidad de registros y capacidad de crecimiento.

1.4 Alcance, restricciones y supuestos

El proyecto se enfoca en el análisis de las ventas y pagos de un restaurante, utilizando la base de datos BD_PUNTOVENTA. Se considera la extracción, transformación y carga de datos mediante ETL, la creación de un Data Warehouse con seis dimensiones y dos tablas de hechos, y la elaboración de indicadores y gráficos en Power BI.

El análisis se limita a la información de pedidos, productos, documentos de venta y formas de pago. No se incluyen otros procesos como inventarios, compras o reservas, ya que no forman parte de los datos disponibles.

Actualmente, se trabaja con una cantidad reducida de registros para realizar las pruebas. Por ello, se plantea una etapa experimental de Big Data que permita evaluar el procesamiento de un mayor volumen de información, considerando los recursos y el tiempo disponibles.

Finalmente, se considera que los datos deben mantener una estructura correcta para poder relacionarlos. Asimismo, se evitará utilizar información personal de los clientes, priorizando la seguridad y privacidad de los datos.

1.5 Stakeholders y criterios de éxito

STAKEHOLDER	NECESIDAD / DECISIÓN	CRITERIO DE ÉXITO	EVIDENCIA
Administrador del restaurante	Conocer el comportamiento general de las ventas	Visualizar ventas totales, cantidades vendidas y evolución por fecha	Dashboard de Power BI
Responsable de operaciones	Identificar productos y personal con mayor movimiento	Obtener productos más vendidos y ventas por mozo	Gráficos del dashboard
Responsable de caja	Revisar la actividad de las cajas y formas de pago	Visualizar ventas por caja y distribución de pagos	DW_FACT_PAGOS y dashboard
Equipo de análisis	Contar con información organizada y sin duplicaciones	Cargar correctamente dimensiones y tablas de hechos	Validación del proceso ETL
Equipo del proyecto	Evaluar el crecimiento de la solución	Ejecutar una prueba con mayor volumen y registrar tiempo y cantidad procesada	Resultados de experimentación Big Data

1.6 Fuentes, gobierno y ética de los datos

La principal fuente de información del proyecto es la base de datos BD_PUNTOVENTA, que contiene los registros de pedidos, productos, ventas y pagos del restaurante. Para el análisis se utilizan las tablas MPEDIDO, DPEDIDO, TPRODUCTO, MDOCUMENTO, DDOCUMENTO y DPAGODOCUMENTO.

Durante el proceso ETL se realizan algunas validaciones, como la selección de datos, el tratamiento de fechas y la eliminación de registros duplicados. Después de la carga, se verifica la cantidad de registros en las dimensiones y tablas de hechos para comprobar que la información se haya cargado correctamente.

En cuanto a la seguridad, se considera utilizar únicamente los datos necesarios para el análisis, evitando mostrar información personal de los clientes. Además, se debe tener en cuenta la calidad de los datos, ya que durante las pruebas se encontraron diferencias entre los montos de ventas y pagos. Por ello, es importante revisar estas diferencias para obtener resultados más confiables.

CAPÍTULO II: ANTECEDENTES Y FUNDAMENTOS

2.1 Antecedentes

Para el desarrollo del proyecto se revisaron investigaciones relacionadas con el análisis de ventas, el uso de datos y la aplicación de Inteligencia de Negocios y Big Data en restaurantes. Estos estudios permiten conocer otras soluciones y tomar como referencia algunas ideas para nuestro trabajo.

Antecedente 1: Pronóstico de demanda en restaurantes

Tanizaki et al. (2019) desarrollaron un estudio para mejorar el pronóstico de demanda en restaurantes mediante Machine Learning. Utilizaron datos reales de establecimientos y consideraron factores como el clima, la ubicación y los eventos. Este trabajo demuestra la importancia de combinar diferentes fuentes de información, algo que podría aplicarse en una futura ampliación de nuestro proyecto.

Antecedente 2: Predicción de ventas mediante Machine Learning

Schmidt et al. (2022) analizaron las ventas de un restaurante utilizando diferentes modelos de Machine Learning para realizar pronósticos diarios y semanales. En algunas pruebas obtuvieron errores de predicción cercanos al 19,5 %. Este estudio demuestra que los datos históricos de ventas pueden utilizarse no solo para elaborar reportes, sino también para realizar predicciones.

Antecedente 3: Análisis de datos internos y externos

Chae et al. (2024) realizaron una investigación utilizando información de ventas y promociones de una cadena de restaurantes entre 2019 y 2022, incorporando también

datos económicos y otras variables externas. Compararon modelos de Machine Learning y Deep Learning para mejorar los pronósticos de demanda. Este antecedente muestra cómo integrar diferentes fuentes puede ayudar a realizar un análisis más completo de las ventas.

Antecedente 4: Predicción de ventas utilizando datos climáticos

Kim (2025) desarrolló un modelo de inteligencia artificial para predecir la demanda de productos de un restaurante, utilizando información histórica de ventas y datos climáticos entre 2021 y 2023. Se analizaron aproximadamente 384 productos mediante redes neuronales, obteniendo una correlación media de 0,7945. Este estudio se relaciona con nuestro proyecto porque ya contamos con dimensiones de productos y fechas, que podrían servir como base para incorporar información climática.

Antecedente 5: Big Data aplicado a restaurantes

Gómez-Talal et al. (2024) realizaron una investigación utilizando información de tickets de venta y variables ambientales para analizar el comportamiento de la demanda en restaurantes. Aplicaron técnicas de Machine Learning no supervisado y métodos estadísticos para identificar relaciones entre productos, ventas y temperatura. Este antecedente aporta ideas para la etapa experimental de Big Data que se plantea desarrollar en nuestro proyecto.

Los estudios revisados muestran que organizar correctamente los datos permite obtener información útil para analizar las ventas. En nuestro proyecto, esta primera parte se desarrolla mediante el proceso ETL, el Data Warehouse y Power BI. Además, estos antecedentes sirven como referencia para la futura etapa experimental de Big Data, donde se busca evaluar un mayor volumen de registros y la incorporación de nuevas fuentes de información.

2.2 Fundamentos de Inteligencia de Negocios

La Inteligencia de Negocios (BI) permite organizar y transformar los datos de una empresa en información útil para analizar sus actividades y tomar mejores decisiones. En nuestro proyecto se aplica mediante la integración de datos, el proceso ETL, el Data Warehouse y Power BI.

Integración de datos

Consiste en reunir información que se encuentra en diferentes tablas. En el restaurante, los datos de ventas, productos, pedidos y pagos están distribuidos en la base de datos BD_PUNTOVENTA, por lo que es necesario relacionarlos para obtener información completa.

ETL

El proceso ETL comprende tres etapas: extracción, transformación y carga. Primero se obtienen los datos de la base operacional; luego se organizan, se revisan las fechas y se evitan duplicados. Finalmente, se cargan al Data Warehouse, comenzando por las dimensiones y después por las tablas de hechos.

Modelado dimensional

Permite organizar la información en dimensiones y tablas de hechos para facilitar su análisis. Según Kimball Group (s. f.), este modelo ayuda a estudiar los procesos del negocio desde diferentes perspectivas.

En nuestro proyecto se utilizan seis dimensiones: fecha, producto, cliente, mozo, caja y forma de pago; además de dos tablas de hechos para ventas y pagos.

Data Warehouse y Data Mart

Un Data Warehouse almacena información organizada para realizar consultas y análisis, mientras que un Data Mart se enfoca en un área específica del negocio. En nuestro caso, el Data Warehouse está orientado al análisis de ventas y pagos del restaurante.

OLAP

OLAP permite analizar los datos desde diferentes perspectivas, como fechas, productos o clientes. Según Kimball Group (s. f.), el esquema estrella facilita este tipo de análisis mediante la relación entre dimensiones y hechos.

Indicadores y visualización

Los indicadores permiten conocer los resultados del negocio. En nuestro proyecto se utilizan Ventas Totales, Cantidad Vendida, Monto Pagos y Propinas.

Estos indicadores se calculan y presentan en Power BI mediante medidas DAX, tarjetas y gráficos que facilitan la interpretación de los resultados (Microsoft, s. f.).

2.3 Fundamentos de Big Data

Big Data se refiere al manejo de grandes cantidades de datos que, por su volumen, velocidad o variedad, pueden requerir nuevas formas de procesamiento.

En nuestro proyecto, los datos actuales son pequeños y pueden procesarse con SQL Server, por lo que Big Data todavía no está implementado. Se plantea como una etapa experimental para evaluar cómo responde la solución cuando aumenta la cantidad de información.

Esta propuesta considera cinco aspectos:

- Volumen: aumentar progresivamente los registros de ventas y pagos para observar los tiempos de procesamiento.

- Velocidad: considerar que los pedidos y pagos se generan continuamente y podrían requerir actualizaciones más frecuentes.
- Variedad: incorporar en una etapa futura fuentes adicionales, como información climática, promociones o días festivos.
- Veracidad: revisar la calidad y consistencia de los datos. En las pruebas actuales se identificó una diferencia entre los totales de ventas y pagos que requiere revisión.
- Escalabilidad: evaluar cómo responde la solución cuando crece la cantidad de registros.

Por ello, Big Data se considera una posible ampliación del proyecto, cuyo propósito será evaluar el procesamiento de un mayor volumen de información y la incorporación de nuevas fuentes de datos.

2.4 Tecnologías, estándares y trabajos relacionados

Para la solución se consideran tecnologías diferentes según la función que cumplen dentro de la arquitectura.

Tecnología	Uso en el proyecto	Ventaja principal	Consideración
SQL Server	Base operacional y Data Warehouse	Integración con SQL y herramientas Microsoft	Adecuado para el volumen actual
SQL / ETL	Extracción, transformación y carga	Fácil integración con las tablas actuales	El procesamiento actual es principalmente por lotes
Power BI	Modelo analítico y dashboard	Integración directa con SQL Server y DAX	Orientado principalmente al análisis y visualización
Apache Spark	Experimentación Big Data	Procesamiento distribuido y escalable	Requiere una infraestructura y configuración adicional

Python/ PySpark	Procesamiento experimental	Permite trabajar con Spark mediante Python	Se utilizará principalmente en la etapa Big Data

SQL Server es adecuado para la implementación actual, ya que los datos están estructurados y su volumen es pequeño. Además, permite gestionar la información y realizar consultas para el análisis (Microsoft, s. f.).

Power BI se mantiene como herramienta de visualización porque ya está integrado con el Data Warehouse. Mediante medidas DAX, se calculan indicadores que cambian según los filtros seleccionados (Microsoft, s. f.).

Para la etapa experimental se propone Apache Spark, que permite procesar grandes volúmenes de datos de forma distribuida y trabajar con información almacenada o flujos de datos (Apache Software Foundation, s. f.).

CAPÍTULO III: DISEÑO DE INGENIERÍA DE LAS SOLUCIONES

3.1 Requerimientos funcionales y no funcionales

La solución se diseñó para transformar la información transaccional de un sistema de gestión de restaurante, inspirado en INFOREST, en información analítica verificable. Se implementaron dos componentes complementarios: una solución de Inteligencia de Negocios basada en un dashboard HTML interactivo y una solución Big Data basada en Apache Spark/PySpark.

Requerimientos funcionales. Los requerimientos funcionales trazan el flujo desde la base operacional hasta el consumo analítico:

Código	Requerimiento funcional	Criterio de verificación
RF-01	Almacenar pedidos, detalle de pedidos, productos, documentos de venta, detalle facturado y pagos en SQL Server.	Existencia y consulta de las seis tablas operacionales.
RF-02	Ejecutar de forma automatizada el proceso ETL hacia el Data Warehouse.	Ejecución de sp_CargarDataWarehouse sin errores.
RF-03	Cargar las dimensiones de fecha, producto, cliente, mozo, caja y forma de pago.	Conteos esperados después del ETL.
RF-04	Registrar las ventas a nivel de producto por documento y los pagos a nivel de medio de pago por documento.	DW_FACT_VENTAS y DW_FACT_PAGOS con granularidades separadas.
RF-05	Calcular KPI de ventas, unidades, documentos, ticket promedio, pagos y propinas.	Valores reconciliados con SQL Server y PySpark.
RF-06	Permitir análisis interactivo por fecha, producto, mozo y caja.	Filtros funcionales en Dashboard_BI_INFOREST.html.

Código	Requerimiento funcional	Criterio de verificación
RF-07	Procesar los datos del Data Warehouse con Apache Spark mediante DataFrames, JOIN, groupBy y agregaciones.	Notebook PySpark ejecutable con resultados.
RF-08	Generar una salida procesada en formato Parquet.	Lectura posterior del Parquet y reconciliación de totales.
RF-09	Evaluar escalabilidad con volúmenes 1x, 5x y 10x, con tres repeticiones por escenario.	Nueve ejecuciones registradas con tiempo y throughput.

Requerimientos no funcionales. Se establecieron metas cuantificables de calidad, rendimiento, reproducibilidad, seguridad y mantenibilidad:

Código	Requerimiento no funcional	Meta / umbral
RNF-01	Completitud de datos críticos.	100 % de registros con fecha, producto, cliente, mozo, caja y monto requeridos.
RNF-02	Consistencia entre ventas y pagos.	Diferencia de reconciliación igual a S/ 0.00 en el dataset base.
RNF-03	Rendimiento del ETL en el dataset académico.	Tiempo promedio menor a 1 segundo.
RNF-04	Escalabilidad del procesamiento Spark.	Procesar 150 000 registros (10x) en menos de 2 segundos en el entorno experimental.
RNF-05	Reproducibilidad.	Backup/script SQL, CSV, notebook PySpark y dashboard HTML conservados como evidencia.
RNF-06	Seguridad y privacidad.	No almacenar credenciales ni datos personales reales en los artefactos del proyecto.
RNF-07	Portabilidad del producto BI.	Dashboard final abrible en navegador sin dependencia de software BI propietario.
RNF-08	Mantenibilidad.	Nombres de tablas y artefactos consistentes, código organizado y reglas de carga documentadas.

3.2 Alternativas y decisiones de arquitectura

Se compararon alternativas considerando integración con SQL Server, facilidad de despliegue, costo académico, reproducibilidad, capacidad de visualización y posibilidad de evaluar escalabilidad. La alternativa seleccionada mantiene el Data Warehouse en SQL Server, genera un dashboard BI portátil con Python/Plotly y utiliza Apache Spark en Google Colab para la solución Big Data.

Alternativa	Criterios	Ventajas	Limitaciones / riesgos	Decisión
SQL Server + Python/Plotly + PySpark/Colab	Integración, costo, portabilidad, reproducibilidad, escalabilidad experimental.	Usa el DW existente; dashboard HTML autónomo; Spark permite pruebas 1x/5x/10x; no requiere servidor adicional.	El dashboard no incluye gobierno multiusuario ni actualización programada; Spark se ejecuta en Colab y no en un clúster productivo.	Seleccionada. Es la arquitectura implementada y validada.
SQL Server + Power BI + PySpark/Colab	Visualización BI, integración con ecosistema Microsoft.	Herramienta BI especializada y amplia capacidad de visualización.	Mayor dependencia del entorno de escritorio/servicio; no mejora la reproducibilidad del experimento Big Data para este proyecto.	No seleccionada para la implementación final.
SQL Server + Python/Pandas/Matplotlib	Simplicidad y bajo costo.	Fácil de programar para volúmenes pequeños.	No demuestra procesamiento distribuido ni escalabilidad con Spark; menor separación entre BI y Big Data.	Descartada como arquitectura final.

3.3 Arquitectura integrada y flujo de datos

La arquitectura integra cuatro capas: base operacional, ETL automatizado, Data Warehouse y consumo analítico. Los datos se originan en SQL Server, se transforman mediante el procedimiento almacenado sp_CargarDataWarehouse y se cargan en un modelo dimensional con dos tablas de hechos. A partir del Data Warehouse se exportan archivos CSV que alimentan tanto el dashboard BI como el notebook PySpark. Spark genera además una salida Parquet procesada.

El límite de la solución BI se encuentra en el consumo analítico del Data Warehouse mediante un dashboard HTML interactivo. El límite de la solución Big Data comienza en la ingesta batch de los CSV y comprende normalización, validaciones, JOIN, agregaciones, experimentación de escalabilidad y almacenamiento Parquet. La interfaz de integración entre ambas soluciones es el mismo Data Warehouse y su exportación controlada a CSV.

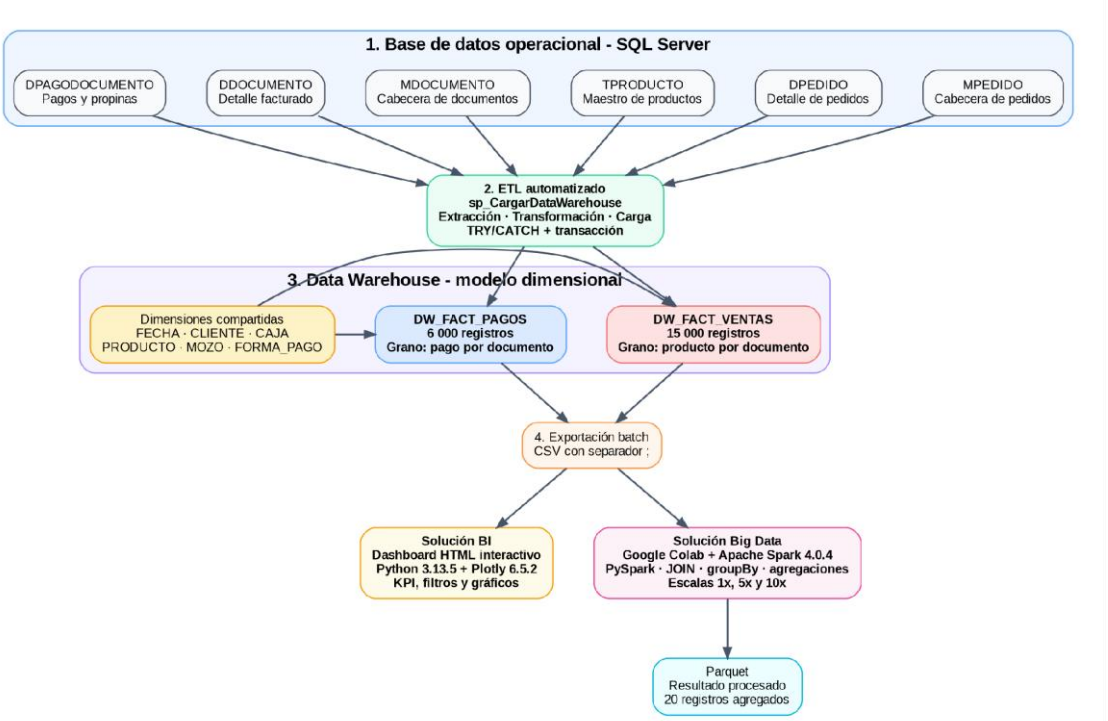


Figura 1. Arquitectura integrada de la solución BI y Big Data. Fuente: elaboración propia.

3.4 Diseño de la solución de Inteligencia de Negocios

3.4.1 Requisitos, usuarios e indicadores

Los usuarios objetivo son el administrador y el supervisor del restaurante. La solución busca apoyar decisiones relacionadas con comportamiento de ventas, desempeño de productos y mozos, distribución por caja y composición de medios de pago. La actualización se realiza en modalidad batch después de ejecutar el ETL y regenerar el artefacto BI.

Pregunta / decisión	Usuario	KPI	Fórmula	Granularidad / fuente	Meta de validación
¿Cuánto se vendió en el periodo?	Administrador	Ventas totales	SUM(venta)	Detalle de venta / DW_FACT_VENTAS	Coincidir con SQL Server: S/ 365 122.
¿Cuántas unidades se vendieron?	Administrador	Unidades vendidas	SUM(cantidad)	Detalle de venta / DW_FACT_VENTAS	17 142 unidades en dataset base.
¿Cuántos documentos se emitieron?	Administrador	Documentos	COUNT DISTINCT(tDocumento)	Documento / DW_FACT_VENTAS	5 000 documentos.
¿Cuál es el valor promedio por documento?	Administrador	Ticket promedio	Ventas / Documentos	Documento	S/ 73.0244.
¿Qué productos y mozos generan mayor venta?	Supervisor	Venta por producto / mozo	SUM(venta) por dimensión	Producto y mozo	Ranking consistente con SQL y Spark.
¿Cómo se distribuyen los pagos?	Administrador	Monto por forma de pago	SUM(monto) por forma de pago	Pago / DW_FACT_PAGOS	Total S/ 365 122.
¿Cuánto se registró en propinas?	Administrador	Propinas	SUM(propina)	Pago / DW_FACT_PAGOS	S/ 1 000.

3.4.2 Modelo dimensional y almacén de datos

El Data Warehouse se diseñó como una constelación de hechos porque las ventas y los pagos poseen granularidades diferentes. DW_FACT_VENTAS representa una línea de producto facturada dentro de un documento, mientras que DW_FACT_PAGOS representa un registro de pago asociado a un documento. Esta separación evita multiplicar importes cuando un documento contiene varios productos y, al mismo tiempo, más de un medio de pago.

Las dimensiones utilizan claves sustitutas enteras (idFecha, idProducto, idCliente, idMozo, idCaja e idFormaPago). Los códigos operacionales se conservan como atributos de negocio. En la versión académica actual no se implementan dimensiones lentamente cambiantes; las dimensiones se cargan como catálogos completos del dataset.

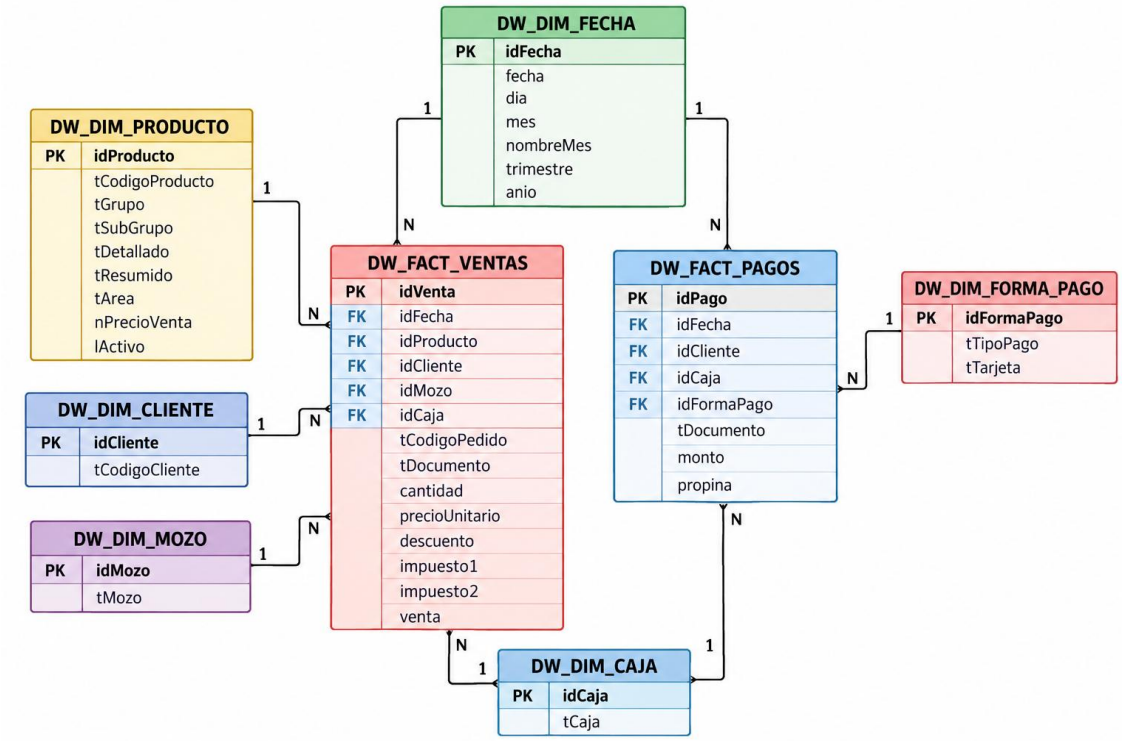


Figura 2. Modelo dimensional implementado para ventas y pagos. Fuente: elaboración propia.

Tabla	Tipo	Granularidad / función	Registros cargados
DW_DIM_FECHA	Dimensión	Día calendario con atributos día, mes, trimestre y año.	180
DW_DIM_PRODUCTO	Dimensión	Catálogo de productos, grupos, subgrupos, área y precio.	20
DW_DIM_CLIENTE	Dimensión	Identificación del cliente asociado al documento.	100
DW_DIM_MOZO	Dimensión	Código del mozo asociado al pedido.	8
DW_DIM_CAJA	Dimensión	Caja donde se registró el documento.	3
DW_DIM_FORMA_PAGO	Dimensión	Tipo de pago y tarjeta cuando aplica.	4
DW_FACT_VENTAS	Hechos	Una fila por producto vendido dentro de un documento.	15 000
DW_FACT_PAGOS	Hechos	Una fila por pago utilizado en un documento.	6 000

3.4.3 Procesos ETL/ELT y calidad

El proceso ETL fue automatizado mediante el procedimiento almacenado sp_CargarDataWarehouse. La extracción toma información de MPEDIDO, DPEDIDO, TPRODUCTO, MDOCUMENTO, DDOCUMENTO y DPAGODOCUMENTO. La transformación selecciona atributos, elimina duplicados cuando corresponde, crea la dimensión fecha, asigna claves sustitutas y relaciona las claves de negocio con las dimensiones. La carga se ejecuta primero sobre dimensiones y después sobre tablas de hechos.

El procedimiento utiliza una transacción junto con TRY...CATCH y XACT_ABORT. Si ocurre una excepción, se ejecuta ROLLBACK para impedir una carga parcial; si todas las operaciones terminan correctamente se confirma la transacción mediante COMMIT. De esta forma la automatización cumple una función de control de integridad además de simplificar la ejecución repetible del ETL.

Tabla operacional	Registros	Control de calidad crítico	Resultado
MPEDIDO	5 000	Pedidos sin mozo	0
DPEDIDO	15 000	Conteo de detalle operativo	15 000
TPRODUCTO	20	Productos maestros	20
MDOCUMENTO	5 000	Sin fecha / cliente / caja	0 / 0 / 0
DDOCUMENTO	15 000	Sin producto / cantidad / venta	0 / 0 / 0
DPAGODOCUMENTO	6 000	Registros de pago	6 000
TOTAL OPERACIONAL	41 020	Periodo analizado	01/01/2026 al 29/06/2026

Tabla Data Warehouse	Registros después del ETL
DW_DIM_FECHA	180
DW_DIM_PRODUCTO	20
DW_DIM_CLIENTE	100
DW_DIM_MOZO	8
DW_DIM_CAJA	3
DW_DIM_FORMA_PAGO	4
DW_FACT_VENTAS	15 000
DW_FACT_PAGOS	6 000
TOTAL	21 315

Para medir la automatización se ejecutó el procedimiento completo tres veces sobre el mismo dataset. Los tiempos registrados fueron 0.381 s, 0.463 s y 0.374 s, con un promedio de 0.406 s. Este resultado satisface el umbral establecido de menos de un segundo para la carga académica base.

Ejecución ETL	Tiempo (ms)	Tiempo (s)
1	381	0.381
2	463	0.463
3	374	0.374
Promedio	406	0.406

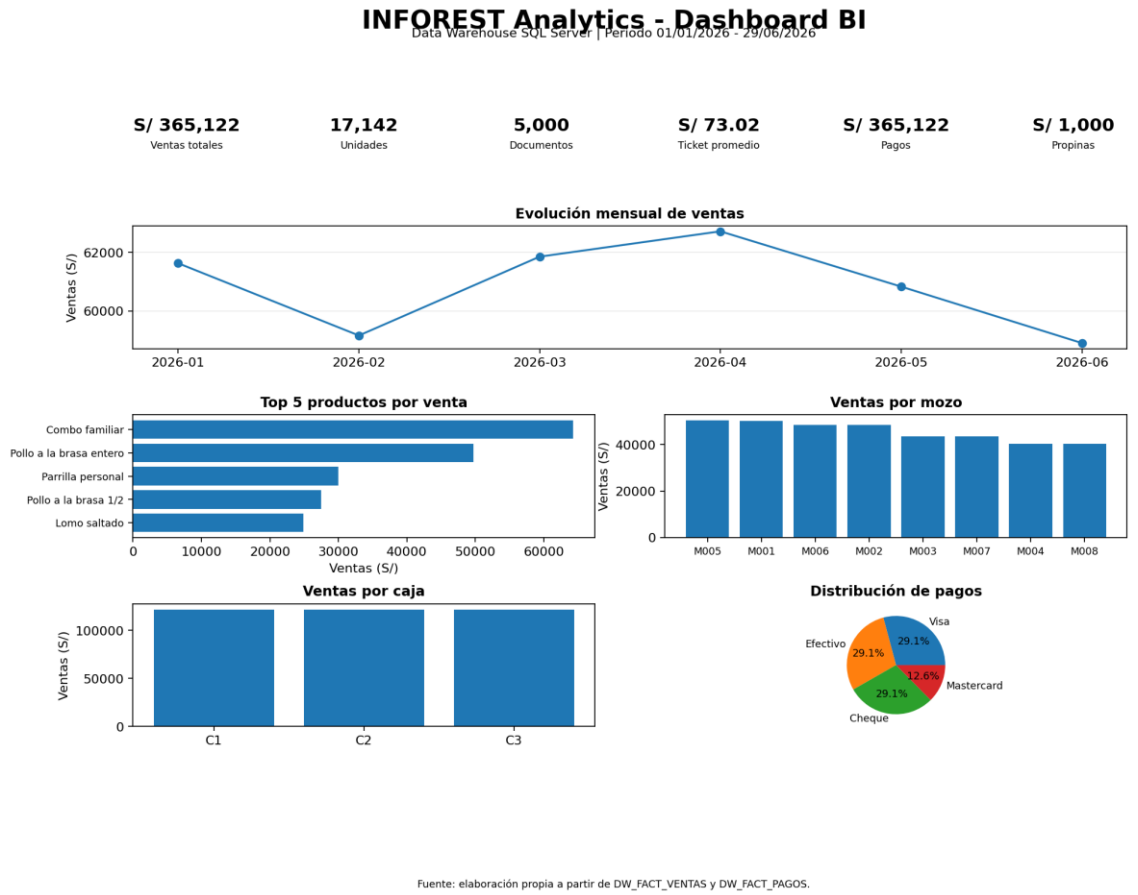
La reconciliación final confirmó 15 000 registros en DW_FACT_VENTAS, 6 000 en DW_FACT_PAGOS, 17 142 unidades vendidas, ventas por S/ 365 122, pagos por S/ 365 122 y propinas por S/ 1 000. No se detectaron claves dimensionales nulas en las dos tablas de hechos y la diferencia entre ventas y pagos fue S/ 0.00, por lo que la reconciliación del dataset base fue del 100 %.

3.4.4 OLAP, reportes y dashboards

La solución BI final se implementó como un dashboard HTML interactivo generado con Python 3.13.5 y Plotly 6.5.2. El archivo Dashboard_BI_INFOREST.html es autónomo y puede abrirse en un navegador moderno sin instalar una plataforma BI propietaria. El dashboard consume los datos exportados desde el Data Warehouse y conserva la separación entre hechos de ventas y hechos de pagos.

Los filtros disponibles son fecha, producto, mozo y caja. Los indicadores de ventas responden a todos los filtros aplicables. Debido a que pagos y ventas tienen granularidades diferentes, la visualización de pagos se filtra por fecha y caja, evitando relacionar pagos con producto o mozo de forma artificial. El diseño incluye tarjetas de KPI, evolución mensual, Top 5 de productos, ventas por mozo, ventas por caja y distribución de pagos.

Las operaciones analíticas implementadas corresponden a filtrado dimensional, agregación, ranking y análisis temporal. No se implementó un cubo MOLAP independiente; las operaciones OLAP se resuelven sobre el modelo dimensional mediante agregaciones del dashboard.



Fuente: elaboración propia a partir de DW_FACT_VENTAS y DW_FACT_PAGOS.

Figura 3. Vista general del dashboard BI implementado. Fuente: elaboración propia a partir del Data Warehouse.

Indicador	Resultado base
Ventas totales	S/ 365 122
Unidades vendidas	17 142
Documentos	5 000
Ticket promedio	S/ 73.02
Pagos totales	S/ 365 122
Propinas	S/ 1 000

3.5 Diseño de la solución Big Data

3.5.1 Justificación y requisitos Big Data

El volumen actual del proyecto no constituye por sí solo un caso empresarial de Big Data: la base operacional contiene 41 020 registros y el Data Warehouse 21 315 registros. Por ello, la solución Big Data se plantea de forma explícita como un prototipo de procesamiento escalable y reproducible. Su propósito es demostrar que la misma lógica analítica puede ejecutarse con Apache Spark y mantenerse consistente cuando el volumen de la tabla de hechos se incrementa de forma controlada.

La evaluación utiliza DW_FACT_VENTAS como dataset principal de escalabilidad. Se prueban 15 000 registros (1x), 75 000 registros (5x) y 150 000 registros (10x). El incremento se realiza mediante replicación controlada exclusivamente para pruebas de rendimiento; estos datos sintéticos no se utilizan para extraer conclusiones comerciales.

3.5.2 Arquitectura y componentes

La solución Big Data se ejecutó en Google Colab con Apache Spark 4.0.4 y PySpark. La ingesta es batch mediante archivos CSV exportados desde el Data Warehouse. El procesamiento emplea DataFrames, JOIN, groupBy, SUM, COUNT y broadcast de la dimensión de productos durante el experimento. La salida procesada se almacena en Parquet. La orquestación es manual mediante notebook, suficiente para el alcance académico; no se implementó streaming ni un clúster multinodo.

Componente	Versión / configuración	Función implementada
Google Colab	Runtime cloud; 2 CPU lógicos; 12.67 GB RAM aprox.	Entorno de ejecución reproducible del notebook.
Apache Spark	4.0.4; master local[*]; paralelismo 2	Motor de procesamiento de DataFrames.
Python	3.13.16 en Colab	Lógica PySpark y medición experimental.
Java / OpenJDK	21.0.12.1	Runtime de la JVM utilizada por Spark.
CSV	Separador ;	Intercambio batch desde el Data Warehouse.
Parquet	Formato columnar de salida	Persistencia del resultado agregado por producto.
Notebook	ProyectoFinal_BigData_PySpark.ipynb	Ingesta, validaciones, análisis y experimento.

3.5.3 Modelo y ciclo de vida de datos

El ciclo de vida implementado sigue la secuencia SQL Server → exportación CSV → DataFrames PySpark → validación y normalización → JOIN y agregaciones → Parquet. Los ocho archivos del Data Warehouse ocupan aproximadamente 1.23 MB en conjunto en la ejecución registrada. Spark validó que los conteos de las ocho tablas coincidieran con SQL Server antes de continuar el procesamiento.

Durante la ingesta se detectó un problema real de calidad: inferSchema convirtió códigos como 01, 0101 y 001 en valores numéricos y eliminó ceros a la izquierda. La transformación corrigió esos campos mediante normalización explícita con lpad y conversión a string. También se comprobó que no existieran valores nulos críticos en las tablas de hechos.

La salida Parquet agrupa las ventas por producto, grupo y subgrupo. El archivo generado contiene 20 registros agregados y, al ser leído nuevamente por Spark, conserva 17 142 unidades y S/ 365 122 en ventas, demostrando consistencia entre la entrada y el resultado procesado.

3.5.4 Escalabilidad, resiliencia y operación

La escalabilidad se evalúa variando únicamente el factor de datos y manteniendo constante la lógica de procesamiento. Antes de las mediciones se realiza un calentamiento del motor Spark. Cada escenario se ejecuta tres veces; antes de cada repetición se limpia la caché de Spark. El tiempo se mide con time.perf_counter y el throughput se calcula como registros procesados por segundo.

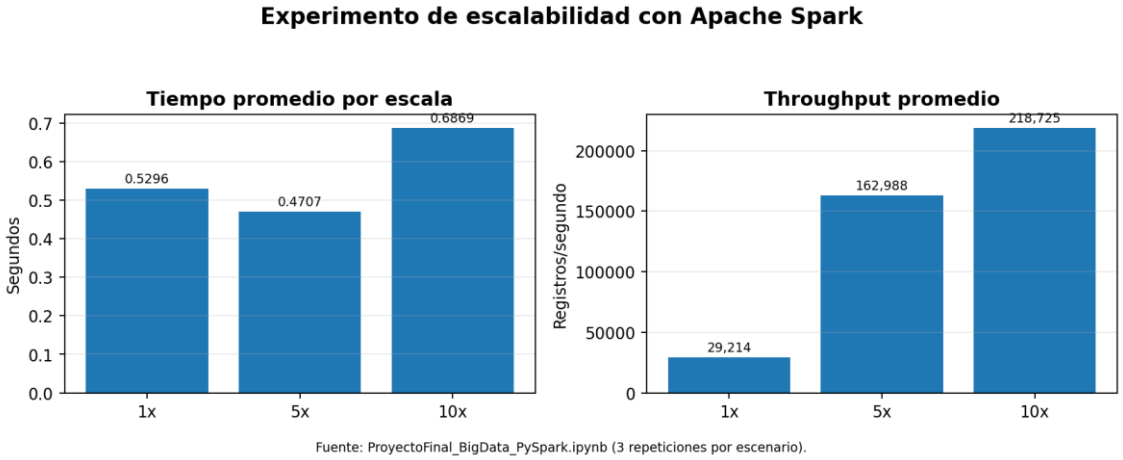
La ejecución se realizó en modo local sobre el runtime de Google Colab. Por tanto, los resultados caracterizan el comportamiento del pipeline en ese entorno y no equivalen a una prueba de escalabilidad horizontal de un clúster multinodo. Spark proporciona mecanismos internos de reejecución de tareas, pero la tolerancia a fallos de nodos no fue sometida a un experimento específico. Como evolución futura se propone ejecutar el mismo notebook sobre un clúster administrado para medir particionamiento y recuperación ante fallos.

3.5.5 Resultados

La solución Spark reprodujo correctamente los KPI obtenidos en SQL Server: 15 000 registros de ventas, 5 000 documentos, 17 142 unidades, S/ 365 122 en ventas; y 6 000 pagos, S/ 365 122 pagados y S/ 1 000 en propinas. La diferencia entre ventas y pagos fue cero. Los Top 5 de productos, las ventas por mozo, por caja y por forma de pago coincidieron con las consultas SQL.

Escala	Registros	Tiempo promedio (s)	Mínimo (s)	Máximo (s)	Desv. estándar (s)	Throughput prom. (reg/s)
1x	15 000	0.5296	0.4102	0.6236	0.1090	29 214.26
5x	75 000	0.4707	0.3853	0.5577	0.0862	162 987.51
10x	150 000	0.6869	0.6504	0.7155	0.0333	218 725.04

En las nueve ejecuciones, las validaciones de número de registros, unidades y ventas resultaron verdaderas. El escenario 10x multiplicó por diez el volumen respecto de 1x y alcanzó un tiempo promedio de 0.6869 s. El escenario 5x registró un tiempo medio menor que 1x; este comportamiento no se interpreta como aceleración causada por el mayor volumen, sino como efecto de la sobrecarga fija, la JVM, la planificación y la variabilidad del entorno compartido de Colab. Por ello se reportan las tres repeticiones y la desviación estándar en lugar de seleccionar solo el mejor tiempo.



Fuente: ProyectoFinal_BigData_PySpark.ipynb (3 repeticiones por escenario).

3.6 Seguridad, privacidad y riesgos

El proyecto utiliza información académica/sintética y códigos de clientes, sin nombres, documentos personales ni credenciales reales. Los principales controles se concentran en integridad, respaldo, manejo de secretos y reproducibilidad. El dashboard se distribuye como archivo HTML local y el notebook no contiene credenciales de SQL Server.

Riesgo	Impacto	Control aplicado / mitigación
Pérdida o corrupción de la base	Imposibilidad de reproducir el ETL.	Conservar backup .bak y script DDL/ETL.
Carga parcial del Data Warehouse	Indicadores inconsistentes.	Transacción, TRY/CATCH, XACT_ABORT y ROLLBACK.
Exposición de credenciales	Acceso no autorizado.	No incluir contraseñas ni cadenas sensibles en notebook o dashboard.
Interpretación errónea de datos sintéticos	Conclusiones comerciales inválidas.	Usar las escalas 5x/10x únicamente para rendimiento.
Alteración de códigos durante inferencia de tipos	Pérdida de ceros a la izquierda.	Normalización explícita de códigos en PySpark.
Variabilidad de Google Colab	Tiempos no deterministas.	Tres repeticiones, promedio, mínimo, máximo y desviación estándar.
Dashboard HTML sin control de acceso empresarial	Uso fuera del ámbito académico.	Distribución local del archivo y uso de datos no sensibles.

3.7 Plan de despliegue y aceptación

El despliegue académico se ejecuta en tres etapas: reconstrucción/carga de SQL Server, generación del producto BI y ejecución de la solución Big Data. Los artefactos principales son el backup o script de la base, el procedimiento sp_CargarDataWarehouse, los CSV del Data Warehouse, Dashboard_BI_INFOREST.html y ProyectoFinal_BigData_PySpark.ipynb.

Paso	Actividad	Criterio de aceptación
1	Restaurar el backup o ejecutar el DDL y verificar las seis tablas operacionales.	Conteos: 5 000 pedidos, 15 000 detalles, 20 productos, 5 000 documentos, 15 000 detalles facturados y 6 000 pagos.
2	Ejecutar sp_CargarDataWarehouse.	21 315 registros totales en dimensiones y hechos; sin error transaccional.
3	Validar KPI y reconciliación.	Ventas = pagos = S/ 365 122; diferencia S/ 0.00; 0 claves críticas nulas.
4	Abrir Dashboard_BI_INFOREST.html.	Seis KPI y visualizaciones disponibles; filtros de fecha, producto, mozo y caja operativos.
5	Ejecutar ProyectoFinal_BigData_PySpark.ipynb.	Conteos iguales a SQL Server, Parquet con 20 registros y reconciliación correcta.
6	Ejecutar experimento 1x/5x/10x.	Nueve validaciones con registros, unidades y ventas = True; escenario 10x menor a 2 s promedio.

El rollback del componente SQL consiste en restaurar el backup y volver a ejecutar el ETL. Los productos analíticos se regeneran a partir de los CSV validados, por lo que no constituyen una fuente primaria de datos. La aceptación final requiere que las métricas del dashboard y PySpark coincidan con SQL Server.

CAPÍTULO IV: DISEÑO DEL EXPERIMENTO

4.1 Pregunta experimental, objetivo e hipótesis

Pregunta experimental: ¿la arquitectura integrada SQL Server → Data Warehouse → dashboard BI / PySpark mantiene la integridad de los indicadores de negocio y un tiempo de procesamiento aceptable cuando el volumen de ventas aumenta de 15 000 a 150 000 registros?

Objetivo experimental: evaluar reproducibilidad, consistencia y escalabilidad de la solución mediante controles de calidad, reconciliación de KPI y mediciones repetidas de tiempo y throughput bajo distintos volúmenes de datos.

Hipótesis H1: la arquitectura conservará el 100 % de consistencia de registros, unidades y ventas en las escalas 1x, 5x y 10x, y el escenario de 150 000 registros será procesado en menos de 2 segundos en el entorno experimental de Google Colab. Hipótesis H0: se producirán diferencias de integridad o el procesamiento del escenario 10x superará el umbral establecido.

4.2 Escenario y datasets

El experimento utiliza datos académicos correspondientes al periodo 01/01/2026 - 29/06/2026. Se distingue el dataset operacional, el Data Warehouse procesado, la exportación para analítica y los conjuntos sintéticos de escalabilidad. La replicación sintética conserva la estructura y los valores del dataset base, pero no representa nuevas ventas reales.

Dataset	Fuente / versión	Volumen / formato	Calidad y privacidad	Uso en BI / Big Data	Evidencia
Original	SQL Server operacional; periodo 01/01/2026-29/06/2026	41 020 filas en 6 tablas relacionales	0 nulos en campos críticos evaluados; datos académicos.	Fuente del ETL.	E-01
Procesado	Data Warehouse posterior a sp_CargarDataWarehouse	21 315 filas en 8 tablas	PK/FK verificadas; hechos sin claves críticas nulas.	Fuente del dashboard y exportación analítica.	E-02
Exportación analítica	8 CSV del DW	≈1.23 MB; separador ;	Conteos reconciliados con SQL; códigos normalizados en Spark.	Entrada del dashboard y notebook.	E-06/E-07
Sintético 1x/5x/10x	Replicación controlada de DW_FACT_VENTAS	15 000 / 75 000 / 150 000 filas	Misma estructura; no se usa para conclusiones comerciales.	Prueba de rendimiento y escalabilidad.	E-03/E-09
Procesado Big Data	Parquet generado por Spark	20 filas agregadas por producto	17 142 unidades y S/ 365 122 reconciliados.	Salida analítica columnar.	E-08

4.3 Variables, métricas y criterios de aceptación

Tipo	Variable / métrica	Definición y unidad	Método de medición	Meta / umbral
Independiente	Factor de escala	1x, 5x y 10x sobre DW_FACT_VENTAS.	crossJoin con factor de replicación controlado.	15 000, 75 000 y 150 000 filas.
Dependiente	Tiempo Spark	Segundos transcurridos por JOIN + groupBy + agregación.	time.perf_counter; 3 repeticiones.	10x < 2 s promedio.
Dependiente	Throughput	Registros procesados por segundo.	Registros / tiempo de ejecución.	Reportar promedio por escala.
Calidad	Completitud crítica	Porcentaje de claves y medidas requeridas no nulas.	Consultas SQL/PySpark.	100 %; 0 nulos críticos.
Calidad	Reconciliación ventas-pagos	Diferencia absoluta entre SUM(venta) y SUM(monto).	SQL Server y PySpark.	S/ 0.00 en dataset base.
Negocio	Consistencia de KPI BI	Ventas, unidades, documentos, ticket, pagos y propinas.	Comparación Dashboard vs SQL.	100 % de coincidencia.
Negocio	Consistencia al escalar	Registros, unidades y ventas esperadas según factor.	Validación booleana en las 9 ejecuciones.	Todas las validaciones = True.
Control	Entorno Spark	Mismo runtime, lógica, calentamiento y limpieza de caché.	Registro del notebook.	Mantener configuración por ejecución.

4.4 Entorno, herramientas y reproducibilidad

Se documentaron dos entornos: el equipo local utilizado para SQL Server/ETL y el runtime de Google Colab utilizado para Spark. El dashboard BI se entrega como HTML autónomo, por lo que su consumo final solo requiere navegador.

Componente	Versión / configuración	Propósito	Evidencia
Sistema operativo local	Microsoft Windows 11 Pro 10.0.26300, x64	Ejecución de SQL Server y SSMS.	E-04
CPU local	Intel Core i5-14400F, 10 núcleos / 16 lógicos	Procesamiento ETL local.	E-04
RAM local	32 GB	Recursos del motor y herramientas locales.	E-04
SQL Server	16.0.1000.6 RTM, Express Edition 64-bit	Base operacional y Data Warehouse.	E-06
SSMS	22.9.1+138.12112.369	Administración y ejecución SQL.	E-06
Dashboard BI	Python 3.13.5 + Plotly 6.5.2; HTML autónomo	Visualización interactiva y KPI.	E-07
Google Colab	2 CPU lógicos, 12.67 GB RAM aprox.	Entorno cloud del experimento Spark.	E-08/E-09
Apache Spark	4.0.4; local[*]; paralelismo 2	Procesamiento Big Data.	E-08
Python Colab	3.13.16	Código PySpark y medición.	E-08
OpenJDK	21.0.12.1	Runtime JVM de Spark.	E-08
Fecha Spark	10/10/2026 22:43:05 (Lima)	Trazabilidad de ejecución.	E-08

La reproducción requiere conservar los archivos CSV, Dashboard_BI_INFOREST.html y ProyectoFinal_BigData_PySpark.ipynb. No se utiliza semilla aleatoria porque el escalamiento replica determinísticamente el dataset base.

4.5 Matriz de escenarios experimentales

Se definieron cuatro escenarios para cubrir la solución BI y la solución Big Data. Los tres escenarios Spark utilizan exactamente la misma operación analítica y tres repeticiones, modificando únicamente el volumen de datos.

ID	Solución	Cambio / configuración	Datos	Métricas	Controles	Resultado esperado
EXP-01	BI	Dashboard HTML sobre el DW base.	15 000 ventas + 6 000 pagos.	6 KPI y filtros.	Mismo DW validado.	KPI coinciden 100 % con SQL Server.
EXP-02	Big Data	Factor 1x; JOIN + groupBy + SUM.	15 000 filas.	Tiempo, throughput, registros, unidades, ventas.	Calentamiento, clearCache, 3 repeticiones.	15 000; 17 142; S/ 365 122; validación True.
EXP-03	Big Data	Factor 5x; misma lógica.	75 000 filas.	Tiempo, throughput y consistencia.	Misma configuración; 3 repeticiones.	75 000; 85 710; S/ 1 825 610; validación True.
EXP-04	Big Data	Factor 10x; misma lógica.	150 000 filas.	Tiempo, throughput y consistencia.	Misma configuración; 3 repeticiones.	150 000; 171 420; S/ 3 651 220; promedio < 2 s.

4.6 Procedimiento reproducible

1. Restaurar el backup de SQL Server o ejecutar el script de creación de la base y verificar las seis tablas operacionales.
2. Ejecutar sp_CargarDataWarehouse y registrar el conteo de las seis dimensiones y las dos tablas de hechos.
3. Validar nulos críticos, cantidad vendida, ventas totales, pagos, propinas y diferencia de reconciliación.
4. Exportar las ocho tablas del Data Warehouse a CSV usando el separador punto y coma (;), conservando una copia versionada de los archivos utilizados.
5. Abrir Dashboard_BI_INFOREST.html y validar que los seis KPI del dataset base coincidan con SQL Server; probar filtros de fecha, producto, mozo y caja.
6. Abrir ProyectoFinal_BigData_PySpark.ipynb en Google Colab e instalar/inicializar PySpark según el notebook.
7. Subir los CSV y ejecutar el control de conteos: 15 000 ventas, 6 000 pagos, 180 fechas, 20 productos, 100 clientes, 8 mozos, 3 cajas y 4 formas de pago.
8. Ejecutar la normalización de códigos para preservar ceros a la izquierda y verificar que las tablas de hechos no contengan valores nulos críticos.
9. Ejecutar los KPI, Top 5 de productos, ventas por mozo/caja y pagos por tipo; comparar con SQL Server.
10. Generar el resultado Parquet, leerlo nuevamente y validar 17 142 unidades y S/ 365 122 en ventas.
11. Realizar el calentamiento del motor Spark, limpiar caché antes de cada medición y ejecutar los factores 1x, 5x y 10x tres veces cada uno.

12. Registrar tiempos, throughput, mínimo, máximo y desviación estándar; ejecutar la validación booleana de registros, unidades y ventas.
13. Exportar resultados_experimento_spark.csv y resumen_experimento_spark.csv y conservarlos junto con el notebook final.
14. Contrastar los resultados con los criterios de aceptación definidos en 4.3 y registrar desviaciones o limitaciones.

4.7 Riesgos, validez y limitaciones del experimento

Riesgo / limitación	Amenaza a la validez	Mitigación / interpretación
Dataset base pequeño	Los 21 315 registros del DW no representan por volumen un entorno Big Data empresarial.	Declarar el alcance como prototipo de escalabilidad y usar 5x/10x solo para rendimiento.
Replicación sintética	Repite patrones existentes y no simula nuevas distribuciones de negocio.	No utilizar los datos escalados para inferencias comerciales.
Google Colab compartido	CPU/RAM y latencia pueden variar entre sesiones.	Tres repeticiones, promedio y desviación estándar; documentar configuración.
Spark en modo local	No valida escalabilidad horizontal ni fallos de nodos.	Limitar conclusiones al runtime usado y proponer clúster multinodo como trabajo futuro.
Sobrecarga fija/JVM	Puede hacer que una escala mayor registre menor tiempo en volúmenes pequeños.	No seleccionar solo el mejor tiempo; reportar todas las ejecuciones y variabilidad.
Inferencia automática de tipos	Puede convertir códigos con ceros a la izquierda.	Normalización explícita con strings/lpad y validación posterior.
Encabezados CSV inconsistentes durante pruebas	Puede provocar pérdida de la primera fila si se interpreta como header.	Validar primera línea/conteos y usar carga explícita según formato.
Dashboard HTML local	No incorpora control de acceso, actualización programada ni colaboración empresarial.	Adecuado al alcance académico; evolución futura a una plataforma BI gobernada.

En consecuencia, el experimento permite sostener conclusiones sobre reproducibilidad, consistencia y rendimiento del pipeline en el entorno documentado, pero no permite afirmar comportamiento causal ni rendimiento equivalente en un clúster productivo de múltiples nodos.

CAPÍTULO V: IMPLEMENTACIÓN Y CONTROL DE EVIDENCIAS

5.1 Estructura y versión del repositorio

•**URL:** https://github.com/LilKxng09/basav-bi-bigdata-restaurante

Estructura del repositorio:

Carpeta	Contenido
/sql	01_ddl_oltp_dw.sql, 02_datos_prueba.sql, 03_etl_dimensiones_hechos.sql, 04_validacion.sql
/power bi	puntoventa.pbix
/bigdata	BigData_PySpark_Restaurante.ipynb, resultados_experimento.csv
/docs	Informe final, diagramas, capturas

Instrucciones de ejecución: (1) restaurar el backup o ejecutar el script SQL y verificar las seis tablas operacionales; (2) ejecutar sp_CargarDataWarehouse; (3) exportar las ocho tablas del DW a CSV; (4) abrir puntoventa.pbix en Power BI Desktop y actualizar la conexión a SQL Server; (5) abrir el notebook en Google Colab, subir los CSV y ejecutar todas las celdas. **Dependencias:** SQL Server 2022 Express y SSMS, Power BI Desktop y Google Colab con Apache Spark 4.0.4 (Big Data).

5.2 Implementación de la solución BI

Componentes implementados

- Base operacional en SQL Server con 6 tablas (MPEDIDO, DPEDIDO, TPRODUCTO, MDOCUMENTO, DDOCUMENTO, DPAGODOCUMENTO) y 41 020 registros del periodo 01/01/2026 al 29/06/2026.
- Data Warehouse en constelación de hechos: 6 dimensiones (fecha, producto, cliente, mozo, caja, forma de pago) y 2 tablas de hechos (DW_FACT_VENTAS y DW_FACT_PAGOS), con 21 315 registros en total.

- ETL automatizado en el procedimiento almacenado sp_CargarDataWarehouse, con transacción, TRY…CATCH, XACT_ABORT y ROLLBACK ante errores.
- Dashboard en Power BI Desktop conectado a SQL Server, con medidas DAX (Ventas Totales, Cantidad Vendida, Monto Pagos y Propinas) y segmentadores por fecha y caja.

Decisiones de diseño

- Dos tablas de hechos separadas, porque ventas (producto por documento) y pagos (medio de pago por documento) tienen granularidades distintas; unirlas multiplicaría los importes.
- Los pagos se filtran solo por fecha y caja, las dimensiones que comparten con las ventas, para no relacionarlos de forma artificial con producto o mozo.
- La actualización es por lotes: se ejecuta el ETL y luego se actualiza el modelo de Power BI.

Resultado de la carga

Capa	Registros
Base operacional (6 tablas)	41 020
Dimensiones (180 / 20 / 100 / 8 / 3 / 4)	315
DW_FACT_VENTAS	15 000
DW_FACT_PAGOS	6 000
Total Data Warehouse	21 315

Limitaciones de la implementación: no se implementaron dimensiones lentamente cambiantes (SCD), el cubo OLAP es lógico (agregaciones sobre el modelo dimensional) y el dashboard no incluye control de acceso ni actualización programada. Además, muestra 4 de los 6 KPI definidos en 3.4.1 (faltan documentos y ticket promedio) y no tiene segmentadores de producto y mozo.

5.3 Implementación de la solución Big Data

La solución se ejecutó en **Google Colab** con **Apache Spark 4.0.4 (PySpark) en modo local** (local[*], paralelismo 2; Python 3.13.16 y OpenJDK 21.0.12.1). El notebook ProyectoFinal_BigData_PySpark.ipynb implementa:

- **Ingesta batch:** lectura de los 8 CSV exportados del Data Warehouse y control de conteos contra SQL Server (15 000 ventas, 6 000 pagos, 180 fechas, 20 productos, 100 clientes, 8 mozos, 3 cajas y 4 formas de pago).
- **Normalización y calidad:** corrección de códigos que inferSchema convertía en números y perdían los ceros a la izquierda (01, 0101, 001), mediante lpad y conversión a string; verificación de que no existan nulos críticos en los hechos.
- **Procesamiento:** DataFrames, JOIN, groupBy, SUM y COUNT, con broadcast de la dimensión de productos; cálculo de los KPI, Top 5 de productos y ventas por mozo, caja y forma de pago.
- **Salida:** resultado agregado por producto, grupo y subgrupo en Parquet (20 registros), releído para validar 17 142 unidades y S/ 365 122.
- **Experimento de escalabilidad:** factores 1x, 5x y 10x sobre DW_FACT_VENTAS (15 000, 75 000 y 150 000 filas), con calentamiento previo, limpieza de caché antes de cada medición y 3 repeticiones por escala (9 ejecuciones).
- **Resultados exportados:** resultados_experimento_spark.csv y resumen_experimento_spark.csv.

Limitaciones: ejecución en un solo nodo (modo local), sin streaming ni prueba de tolerancia a fallos; las escalas 5x y 10x se obtienen por replicación controlada y se usan solo para rendimiento.

Evidencias: notebook (E-06, E-08), datos escalados (E-03), comparación de escalas (E-09).

5.4 Calidad, seguridad y pruebas

ID	Requisito	Entrada / condición	Resultado esperado	Resultado	Evidencia
T-01	RF-01	Consulta de las 6 tablas operacionales	5 000 / 15 000 / 20 / 5 000 / 15 000 / 6 000 registros	Cumple: 41 020 registros	E-06
T-02	RF-02, RF-03, RF-04	Ejecutar sp_CargarDataWarehouse	21 315 registros sin error transaccional	Cumple	E-06
T-03	RNF-03	3 ejecuciones completas del ETL	Tiempo promedio < 1 s	Cumple: 0,406 s (381, 463 y 374 ms)	E-06
T-04	RNF-01	Nulos en campos críticos y claves de hechos	0 nulos	Cumple: 0 pedidos sin mozo, 0 documentos sin fecha, cliente o caja, 0 claves nulas en hechos	E-06
T-05	RNF-02	Ventas vs. pagos	Diferencia de S/ 0,00	Cumple: S/ 365 122 = S/ 365 122	E-07
T-06	RF-05, RF-06	KPI del dashboard Power BI vs. SQL Server	100 % de coincidencia en los KPI mostrados	Parcial: [COMPLETAR: tras actualizar el .pbix, confirmar ventas S/ 365 122, 17 142 unidades, pagos S/ 365 122 y propinas S/ 1 000]; el dashboard muestra 4 de 6 KPI y filtros por fecha y caja	E-07

T-07	RF-07	KPI, Top 5, mozo, caja y forma de pago en Spark vs. SQL	Mismos resultados	Cumple	E-08
T-08	RF-08	Lectura del Parquet generado	20 registros; 17 142 unidades; S/ 365 122	Cumple	E-08
T-09	RF-09	9 ejecuciones (1x, 5x, 10x; 3 repeticiones)	Registros, unidades y ventas = True	Cumple: 9 de 9 validaciones verdaderas	E-08, E-09
T-10	RNF-04	Escala 10x (150 000 filas)	Promedio < 2 s	Cumple: 0,6869 s	E-09
T-11	RNF-06	Revisión de artefactos	Sin credenciales ni datos personales	Cumple: el notebook no contiene credenciales y el .pbix no almacena contraseñas; datos académicos con códigos de cliente	E-04
T-12	RNF-05	Conservación de artefactos	Script o backup, CSV, notebook y dashboard HTML	[COMPLETAR: Cumple al subir los archivos finales al repositorio]	E-04

5.5 Registro de retroalimentación y mejoras

Fecha	Retroalimentación	Acción / decisión	Evidencia antes-después	Resp.
10/10/2026	Revisión frente a la plantilla del curso: la solución Big Data	Se implementó y ejecutó la solución Spark con escalas 1x, 5x y 10x	Antes: Big Data solo declarado. Después: 9 ejecuciones validadas (6.3)	Equipo
10/10/2026	Revisión técnica del script SQL (con apoyo de IA): ETL no automatizado, no idempotente y datos de prueba iniciales con diferencia ventas-pagos	Se automatizó la carga en sp_CargarDataWarehouse (transacción y TRY…CATCH) y se amplió el dataset	Antes: 9 ventas y 6 pagos (150 vs. 174). Después: 15 000 y 6 000 (S/ 365 122 = S/ 365 122)	Equipo

5.6 Contribuciones individuales

Integrante	Responsabilidades	Commits / artefactos	Validación del equipo	%
Arroyo Luna, David Orestes	Solución de inteligencia de negocios en Power BI redacción de los capítulos V y VI	puntoventa.pbix; notebook BigData_PySpark_Restaurante.ipynb; capítulos V y VI	Reparto de capítulos acordado en el chat del equipo	100 %

Integrante	Responsabilidades	Commits / artefactos	Validación del equipo	%
Chavarry Miranda, Brenda Fiorella	Proceso ETL (carga de dimensiones y hechos); redacción de los capítulos VII y VIII	Scripts ETL; capítulos VII y VIII;	Reparto de capítulos acordado en el chat del equipo	100 %
Lucas Marmolejo, Erick Giordano	Base de datos operacional y modelo dimensional; redacción de los capítulos III y IV	Scripts DDL de BD_PUNTOVENTA y del DW; capítulos III y IV;	Reparto de capítulos acordado en el chat del equipo	100 %
Taipe Figueroa, Jhon Victor	Informe de trabajos de campo, redacción de los capítulos I y II	Capítulos I y II;	Reparto de capítulos acordado en el chat del equipo	100 %

5.7 Declaración de uso responsable de inteligencia artificial

Herramienta	Finalidad	Contenido utilizado	Verificación humana	Riesgos / privacidad
Claude (Anthropic)	Revisión del script SQL, generación del notebook PySpark de datos sintéticos y borrador de los capítulos V y VI	Código SQL, estructura del informe, datos de prueba	Ejecución del notebook en Colab, contraste de cifras con el dashboard y los scripts, corrección de errores	Solo se compartieron datos de prueba y sintéticos, sin datos personales; el borrador requiere revisión del equipo

CAPÍTULO VI: EJECUCIÓN, RESULTADOS Y ANÁLISIS DE DATOS

6.1 Validación del dataset original y procesado

Control	Original (OLTP)	Procesado (DW / Big Data)	Regla / umbral	Resultado y evidencia
Registros	41 020 en 6 tablas (DDOCUMENTO 15 000; DPAGODOCUMENTO 6 000)	DW 21 315; DW_FACT_VENTAS 15 000; DW_FACT_PAGOS 6 000	Un registro de hecho por cada detalle origen	Cumple (E-01, E-02)
Nulos críticos	0 pedidos sin mozo; 0 documentos sin fecha, cliente o caja; 0 detalles sin producto, cantidad o venta	0 claves dimensionales nulas en los hechos	100 % de completitud	Cumple (E-06)
Reconciliación de montos	Ventas S/ 365 122	Pagos S/ 365 122; diferencia S/ 0,00	Diferencia = 0	Cumple (E-07)
Consistencia SQL vs. Spark	Conteos de 8 tablas en SQL Server	Mismos conteos y KPI en Spark	100 % de coincidencia	Cumple (E-08)

6.2 Registro de ejecuciones

Ejecución	Versión / fecha	Escenario	Configuración	Estado / log
RUN-01	Anterior al 10/10/2026	EXP-01	sp_CargarDataWarehouse, 3 ejecuciones en SQL Server	Exitoso: 381, 463 y 374 ms; 21 315 registros
RUN-02	Anterior al 10/10/2026	EXP-01	Reconciliación de KPI en SQL Server	Exitoso: S/ 365 122 = ventas = pagos
RUN-03	10/10/2026 (Lima); ProyectoFinal_BigData_PySpark.ipynb	EXP-02	Spark 4.0.4 local[*]; 1x = 15 000 filas; 3 repeticiones	Exitoso: validaciones = True
RUN-04	10/10/2026 22:43 (Lima); ProyectoFinal_BigData_PySpark.ipynb	EXP-03	5x = 75 000 filas; 3 repeticiones	Exitoso: validaciones = True
RUN-05	10/10/2026 22:43 (Lima); ProyectoFinal_BigData_PySpark.ipynb	EXP-04	10x = 150 000 filas; 3 repeticiones	Exitoso: validaciones = True

6.3 Tabla consolidada de resultados

Unidades: segundos y registros por segundo. Se reportan las tres repeticiones por escala resumidas en promedio, mínimo, máximo y desviación estándar, sin excluir valores atípicos.

Escenario	Solución	Escala / config.	Latencia (promedio; mín.–máx.; desv.)	Throughput (reg/s)	Recursos / calidad	Evid.

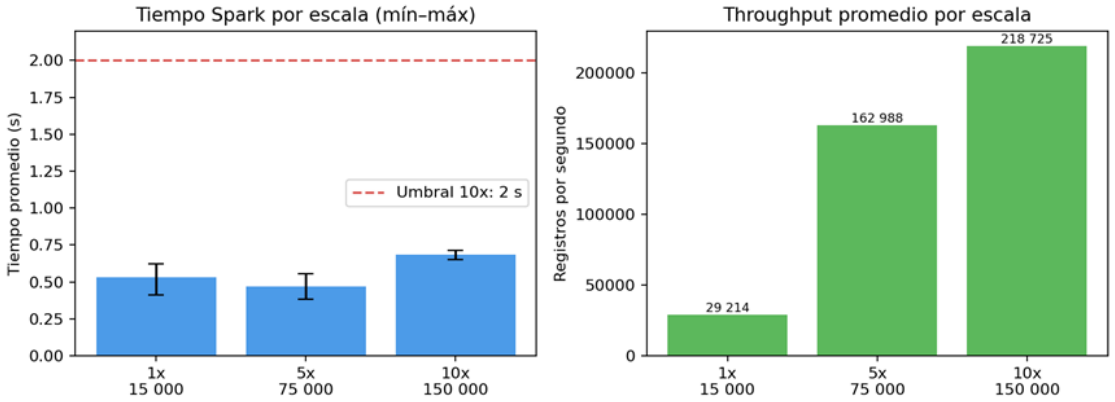
EXP-01	**BI**	**ETL completo sobre 21 315 registros; 3 ejecuciones**	**0,406 s (0,374–0,463)**	**≈ 52 500 (derivado: 21 315 / 0,406)**	**SQL Server 2022 Express; diferencia ventas-pagos S/ 0,00**	**E-06, E-07**
EXP-02	**Big Data**	**1x: 15 000 filas**	**0,5296 s; 0,4102–0,6236; ± 0,1090**	**29 214,26**	**Colab: 2 CPU, 12,67 GB RAM; validaciones True**	**E-08**
EXP-03	**Big Data**	**5x: 75 000 filas**	**0,4707 s; 0,3853–0,5577; ± 0,0862**	**162 987,51**	**Igual configuración; validaciones True**	**E-08, E-09**
EXP-04	**Big Data**	**10x: 150 000 filas**	**0,6869 s; 0,6504–0,7155; ± 0,0333**	**218 725,04**	**Igual configuración; validaciones True**	**E-08, E-09**

6.4 Visualizaciones y dashboards

Visualización	Fuente	Qué decisión apoya	Evidencia
Tarjetas: Ventas Totales, Cantidad Vendida, Monto Pagos y Propinas	DW_FACT_VENTAS, DW_FACT_PAGOS	Panorama general del periodo	E-10
Ventas por fecha	DW_DIM_FECHA	Tendencia y estacionalidad	E-10
Productos más vendidos	DW_DIM_PRODUCTO	Rotación e inventario	E-10
Ventas por mozo	DW_DIM_MOZO	Desempeño del personal de atención	E-10
Ventas por caja	DW_DIM_CAJA	Comparación entre cajas	E-10

Monto por forma de pago (gráfico circular)	DW_DIM_FORMA_PAGO	Medios de pago preferidos	E-10
Tiempo y throughput de Spark por escala (Figura 1)	resultados_experimento_spark.csv	Comportamiento al escalar el volumen	E-09

Figura 1. Tiempo promedio (con mínimo y máximo) y throughput de Spark por escala.





6.5 Análisis e interpretación técnica

1. • **Frente a los objetivos:** el flujo SQL Server → ETL → Data Warehouse → dashboard conservó la integridad de los datos: 0 nulos críticos y reconciliación de S/ 365 122 entre ventas y pagos. Spark reprodujo los mismos KPI y rankings que SQL Server, lo que respalda H1 en consistencia.
2. • **Automatización del ETL:** sp_CargarDataWarehouse cargó 21 315 registros en 0,406 s promedio, por debajo del umbral de 1 s (RNF-03), con variación pequeña entre ejecuciones (0,374 a 0,463 s). La transacción con ROLLBACK evita cargas parciales.
3. • **Escalabilidad Spark:** con diez veces más datos (15 000 a 150 000 filas), el tiempo promedio solo subió de 0,5296 s a 0,6869 s (1,3 veces), mientras el throughput pasó de 29 214 a 218 725 registros por segundo (7,5 veces). El escenario de 150 000 filas quedó muy por debajo del umbral de 2 s (RNF-04).
4. • **Comportamiento no monótono:** el escenario 5x (0,4707 s) resultó más rápido que 1x (0,5296 s). Los rangos de ambas escalas se solapan (0,41–0,62 s frente a 0,39–0,56 s), por lo que no se interpreta como aceleración por mayor volumen, sino como efecto de la sobrecarga fija de Spark, la JVM y la variabilidad del entorno compartido de Colab. Con volúmenes pequeños el tiempo está dominado por costos fijos, y el throughput crece porque esos costos se amortizan.
5. • **Calidad de datos:** el único error real fue la pérdida de ceros a la izquierda por inferSchema, detectado por las validaciones y corregido con lpad. Ilustra la necesidad de cargar explícitamente los tipos de los códigos.
6. • **Límites de las inferencias:** las escalas 5x y 10x replican de forma determinista el dataset base, por lo que no representan nuevas ventas ni nuevas distribuciones; solo sirven para rendimiento. Spark se ejecutó en un solo nodo con 3 repeticiones por escala, y no se midió el mismo cálculo en SQL Server, así que no se puede afirmar que Spark sea más rápido que SQL Server ni extrapolar a un clúster. Con 150 000 filas el volumen sigue siendo pequeño para un caso empresarial de Big Data.

6.6 Verificación de funcionalidad y completitud

Solución BI	RF-01 a RF-06; RNF-01, RNF-02 y RNF-03	Base y DW cargados (41 020 y 21 315 registros), ETL en 0,406 s, ventas y pagos reconciliados en SQL Server y dashboard Power BI conectado al DW; muestra 4 de 6 KPI y filtros por fecha y caja	Parcial (RF-05 y RF-06)	E-07, E-13
Solución Big Data	RF-07 a RF-09; RNF-04	Spark reproduce los KPI de SQL Server, genera Parquet de 20 registros y cumple las 9 validaciones; 10x en 0,6869 s	Cumple (modo local)	E-08, E-14
Integración / despliegue	RNF-05: script o backup, CSV, notebook y dashboard conservados	Artefactos ejecutados; falta subir al repositorio los archivos finales y crear la etiqueta v1.0-final	Parcial	E-04, E-13, E-14

CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES

7.1 Matriz objetivo-resultado-conclusión-recomendación

OBJETIVO	RESULTADO / MÉTRICA	CONCLUSIÓN TÉCNICA	RECOMENDACIÓN / DECISIÓN	EVIDENCIA
OE-01	5 tablas operacionales identificadas (TPRODUCTO, MPEDIDO, MDOCUMENTO, DDOCUMENTO, DPAGODOCUMENTO) cubren el 100% de los datos requeridos.	La estructura normalizada de BD_PUNTOVENTA permitió trazar con precisión el origen de cada dimensión y tabla de hechos, sin necesidad de fuentes externas.	Documentar un diccionario de datos formal de las tablas operacionales para facilitar ampliaciones futuras del modelo.	E-11 / E-07
OE-02	Modelo dimensional tipo estrella con 6 dimensiones y 2 tablas de hechos (DW_FACT_VENTAS,	Separar ventas y pagos en dos tablas de hechos evitó la duplicación de registros, ya que un documento puede	Evaluar un esquema copo de nieve para DW_DIM_PRODUCTO (grupo/subgrupo) si el catálogo de productos crece.	E-11 / E-07

	DW_FACT_PAGOS).	cancelarse con más de una forma de pago.		
OE-03	Proceso ETL ejecutado sin incidencias: 9 registros cargados en DW_FACT_VENTAS y 6 en DW_FACT_PAGOS, más los registros de cada dimensión.	El uso de DISTINCT para eliminar duplicados y la carga de dimensiones antes que hechos garantizó la integridad referencial del Data Warehouse.	Automatizar el proceso ETL mediante un job programado (p. ej. SQL Server Agent) en lugar de una ejecución manual, para sostener cargas periódicas.	E-11 / E-07
OE-04	6 dimensiones y 2 tablas de hechos construidas y relacionadas mediante claves primarias y foráneas (idFecha, idProducto, idCliente, idMozo, idCaja, idFormaPago).	El modelo sigue las buenas prácticas de modelado dimensional (Kimball), lo que facilita consultas analíticas multidimensionales desde Power BI.	Incorporar manejo de dimensiones de cambio lento (SCD) si en el futuro cambian atributos de producto o cliente.	E-11 / E-07
OE-05	Validación mediante consulta SQL (UNION ALL + COUNT) que confirmó la cantidad de registros cargados en cada tabla del Data Warehouse.	El volumen de prueba (datos de ejemplo) permitió validar manualmente, pero sería insuficiente para detectar inconsistencias en un volumen real de producción.	Implementar reglas de calidad automatizadas (sumas de control, detección de nulos y duplicados) antes de escalar a datos reales del restaurante.	E-11 / E-07
OE-06	Dashboard en Power BI con indicadores de ventas totales, cantidad vendida,	La información del Data Warehouse permitió construir visualizaciones que responden	Añadir segmentadores interactivos (rango de fechas, mozo, caja) para que el dashboard	E-11 / E-07

	monto de pagos y propinas, y gráficos por producto, fecha, mozo, caja y forma de pago.	directamente a las preguntas de negocio planteadas, confirmando la utilidad del modelo diseñado.	soporte análisis ad-hoc sin depender de nuevas consultas SQL.	

7.2 Conclusiones técnicas

1. El modelo dimensional de tipo estrella, compuesto por seis dimensiones (DW_DIM_FECHA, DW_DIM_PRODUCTO, DW_DIM_CLIENTE, DW_DIM_MOZO, DW_DIM_CAJA y DW_DIM_FORMA_PAGO) y dos tablas de hechos (DW_FACT_VENTAS y DW_FACT_PAGOS), permitió organizar la información operacional del restaurante de forma coherente con las buenas prácticas de modelado dimensional, evidenciado en la correcta carga de 9 registros de ventas y 6 registros de pagos durante la prueba del proceso ETL.

2. La decisión de separar las ventas y los pagos en dos tablas de hechos independientes resultó correcta desde el punto de vista técnico, ya que un mismo documento de venta puede cancelarse con más de una forma de pago; de haberse unificado en una sola tabla de hechos, se habrían generado registros duplicados que distorsionarían indicadores como el total de ventas.

3. El proceso ETL, apoyado en sentencias INSERT INTO … SELECT, DISTINCT para el control de duplicados y un orden de carga que prioriza las dimensiones sobre las tablas de hechos, garantizó la integridad referencial del Data Warehouse sin generar errores de clave foránea durante las pruebas realizadas.

4. El dashboard construido en Power BI, con indicadores de ventas totales, cantidad vendida, monto de pagos y propinas, además de gráficos por producto, fecha, mozo, caja y forma de pago, demuestra que el modelo dimensional diseñado responde directamente a las preguntas de negocio planteadas en los objetivos específicos del proyecto.

5. El volumen de datos utilizado en las pruebas (9 y 6 registros en las tablas de hechos) es adecuado para validar la lógica del modelo y del proceso ETL, pero es insuficiente para extraer conclusiones de negocio generalizables sobre el comportamiento real de ventas del restaurante.

7.3 Recomendaciones

1. Automatizar el proceso ETL (prioridad alta, impacto alto): programar la ejecución periódica de la extracción, transformación y carga mediante un job de SQL Server Agent

u otra herramienta de orquestación, en lugar de ejecutar los scripts de forma manual, para que el Data Warehouse se mantenga actualizado sin intervención constante del equipo.

2. Incorporar controles de calidad de datos automatizados (prioridad alta, impacto medio): sumar validaciones de conteos, sumas de control y detección de nulos o duplicados antes de la carga, de modo que los errores se detecten antes de llegar al Data Warehouse y no únicamente mediante una verificación manual posterior.

3. Ampliar el dashboard con segmentadores interactivos (prioridad media, impacto medio): agregar filtros por rango de fechas, mozo y caja en Power BI, de manera que los usuarios de negocio puedan explorar la información sin depender de nuevas consultas SQL para cada análisis.

4. Evaluar la incorporación de dimensiones de cambio lento -SCD- (prioridad baja, impacto medio): si en el futuro cambian atributos como el precio o la categoría de un producto, el modelo actual no conserva el historial de esos cambios, lo cual podría afectar el análisis histórico de ventas.

CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL

8.1 Tratamiento ético, legal y seguro de los datos

La solución desarrollada utiliza información proveniente de la operación real de un restaurante, incluyendo datos potencialmente sensibles como el código de cliente (DW_DIM_CLIENTE), el código del mozo asociado a cada venta (DW_DIM_MOZO) y los montos y medios de pago utilizados (DW_FACT_PAGOS, DW_DIM_FORMA_PAGO, incluyendo tipo de tarjeta). Aunque en esta etapa del proyecto se trabajó únicamente con datos de prueba, de escalarse a datos reales de producción sería necesario: (1) contar con autorización del propietario del negocio para el uso de la información con fines académicos o analíticos; (2) evitar almacenar en el Data Warehouse datos de tarjeta más allá del tipo general (evitando números de tarjeta completos u otra información que permita identificar el instrumento de pago); (3) restringir el acceso al Data Warehouse y al dashboard de Power BI mediante roles, de forma que solo el personal autorizado pueda ver información de ventas por mozo o por cliente; y (4) definir una política de retención

8.2 Impactos de las soluciones

Impacto organizacional: la solución permite a la administración del restaurante tomar decisiones basadas en evidencia -por ejemplo, identificar los productos más vendidos, las fechas de mayor demanda o el desempeño asociado a cada mozo y caja- en lugar de apreciaciones informales, lo cual puede mejorar la gestión de inventario, la programación de personal y las estrategias de venta.

Impacto económico: al tratarse de una solución basada en herramientas ya disponibles (motor de base de datos y Power BI), la inversión adicional requerida es relativamente baja en comparación con el valor potencial de la información generada para la toma de decisiones.

Impacto social y riesgos de interpretación: si los indicadores del dashboard se generalizan a partir de un volumen de datos de prueba pequeño (9 registros de ventas y 6 de pagos), existe el riesgo de que se tomen decisiones de negocio -como premiar o cuestionar el desempeño de un mozo- basadas en una muestra no representativa. Es responsabilidad del equipo y de los futuros usuarios de la herramienta validar que los indicadores se calculan sobre un volumen de datos suficiente antes de usarlos para evaluar personas o procesos.

Impacto ambiental: el impacto directo es mínimo; de forma indirecta, la digitalización del análisis de ventas reduce el uso de reportes impresos frente a un esquema de reportería manual.

8.3 Compromiso profesional

El equipo asume la responsabilidad de que el modelo dimensional, el proceso ETL y el dashboard presentados reflejan fielmente la lógica de negocio descrita (ventas y pagos de un restaurante), reconociendo como límite de competencia que las pruebas se realizaron sobre un volumen reducido de datos de ejemplo y no sobre datos de producción en un entorno real de operación continua. El equipo se compromete a mantener la transparencia sobre esta limitación ante el docente y a no presentar los resultados de la prueba (9 y 6 registros) como si fueran representativos del volumen real de ventas del restaurante. Asimismo, se garantiza la reproducibilidad del proceso ETL mediante los scripts SQL documentados (extracción, transformación y carga) y se declara el compromiso de no manipular los datos ni los indicadores del dashboard para presentar resultados más favorables de los que efectivamente arroja el proceso.

REFERENCIAS BIBLIOGRÁFICAS

Chae, B. K., Sheu, C., & Park, E. O. (2024). The value of data, machine learning, and deep learning in restaurant demand forecasting: Insights and lessons learned from a large restaurant chain. Decision Support Systems, 184, 114291. https://doi.org/10.1016/j.dss.2024.114291

Gómez-Talal, I., González-Serrano, L., Talón-Ballestero, P., & Rojo-Álvarez, J. L. (2024). Big data in restaurant management: Unsupervised modelling of ticket data and environmental variables for sales forecasting. En Tourism and ICTs: Advances in data science, artificial intelligence and sustainability. Springer. https://doi.org/10.1007/978-3-031-52607-7_15

Kim, S. (2025). Development of an AI-based restaurant menu demand prediction model utilizing sales and meteorological data. Food Science and Biotechnology. https://doi.org/10.1007/s10068-025-01956-2

Schmidt, A., Kabir, M. W. U., & Hoque, M. T. (2022). Machine learning based restaurant sales forecasting. Machine Learning and Knowledge Extraction, 4(1), 105–130. https://doi.org/10.3390/make4010006

Tanizaki, T., Hoshino, T., Shimmura, T., & Takenaka, T. (2019). Demand forecasting in restaurants using machine learning and statistical analysis. Procedia CIRP, 79, 679–683. https://doi.org/10.1016/j.procir.2019.02.042

ABET. (2026). Criteria for Accrediting Engineering Programs, 2026-2027. https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2026-2027/

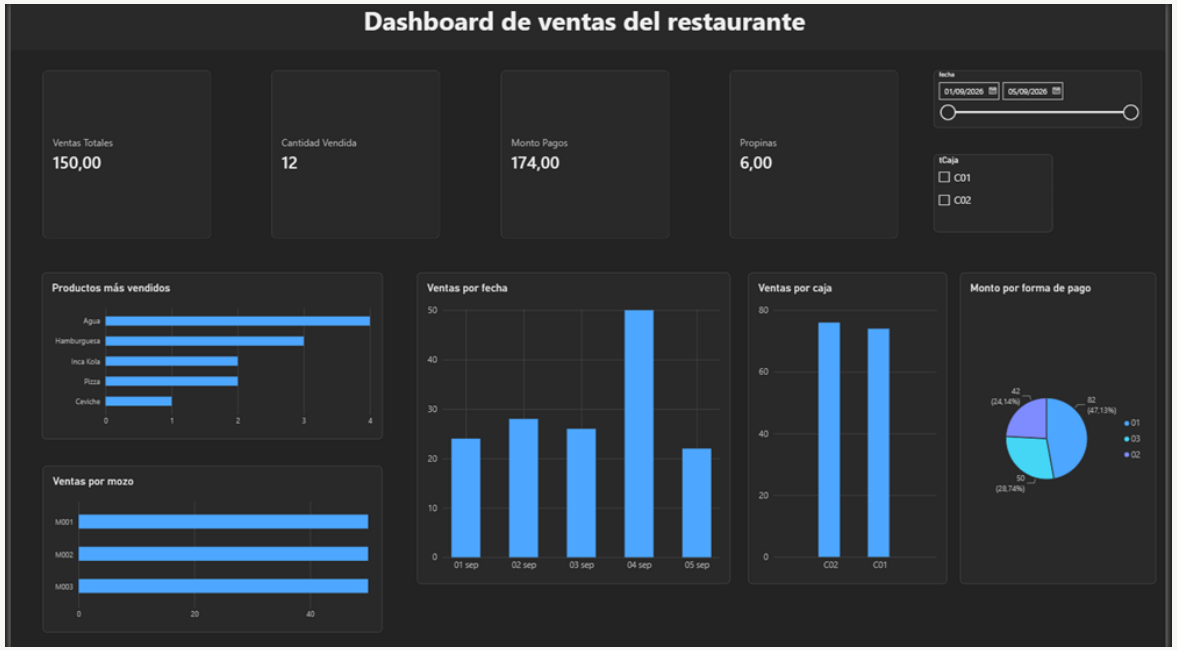
ABET. (s. f.). Assessment Planning. https://www.abet.org/accreditation/get-accredited/assessment-planning/

Apache Software Foundation. (s. f.). Structured Streaming programming guide. https://spark.apache.org/docs/latest/streaming/index.html

Microsoft. (s. f.-a). Ediciones y características admitidas de SQL Server 2022. Microsoft Learn. https://learn.microsoft.com/es-es/sql/sql-server/editions-and-components-of-sql-server-2022?view=sql-server-ver17

Microsoft. (s. f.-b). ¿Qué es Power BI? Microsoft Learn. https://learn.microsoft.com/es-es/power-bi/fundamentals/power-bi-overview

Diseño e implementación de una solución BI basada en ETL y Data Warehouse para el análisis de ventas de un restaurante



Arroyo Luna, David Orestes
Chavarry Miranda, Brenda Fiorella
Lucas Marmolejo, Erick Giordano
Taipe Figueroa, Jhon Victor

CAPÍTULO I

Contexto, problema y objetivos

De datos operacionales dispersos a información integrada para apoyar decisiones del restaurante.

SITUACIÓN ACTUAL

Datos diarios del restaurante

Pedidos · productos · ventas · clientes · pagos

- MPEDIDO
- DPEDIDO
- TPRODUCTO
- MDOCUMENTO
- DDOCUMENTO
- DPAGODOC.

Riesgo

Cruzar ventas y pagos directamente puede duplicar registros y distorsionar los resultados.

Problema de ingeniería

La información está distribuida y tiene distintos niveles de detalle; esto dificulta obtener una visión integrada y confiable para la toma de decisiones.

Solución propuesta

ETL
Extraer, transformar y validar

→

Data Warehouse
6 dimensiones + 2 hechos

→

Power BI
KPIs y visualizaciones

Objetivo central

- Integrar datos de ventas y pagos.
- Analizar por fecha, producto, cliente, mozo, caja y pago.
- Evaluar escalabilidad con una prueba de mayor volumen.

Alcance y criterios

- ✓ Ventas y pagos
- ✓ Privacidad del cliente
- ✓ Validación ETL

CAPÍTULO II

Antecedentes y fundamentos

La propuesta se apoya en BI para el análisis actual y plantea Big Data como evolución experimental.

Antecedentes: ¿qué aportan al proyecto?

Tanizaki et al.	2019	**Schmidt et al.**	2022	**Chae et al.**	2024	**Gómez-Talal et al.**	2024
Demanda + clima/eventos		Pronóstico diario/semanal		Fuentes internas + externas		Big Data + variables ambientales	

Kim — 2025 — IA + ventas + clima

ACTUAL

Fundamentos de Inteligencia de Negocios

Integración	Relacionar tablas
ETL	Extraer · transformar · cargar
Modelo dimensional	6 dimensiones + 2 hechos
Power BI	DAX · tarjetas · gráficos
KPIs del dashboard	Ventas totales · Cantidad vendida · Pagos · Propinas

Big Data

Etapa experimental, no implementación actual

- V Volumen
- V Velocidad
- V Variedad
- V Veracidad
- **Escalabilidad**

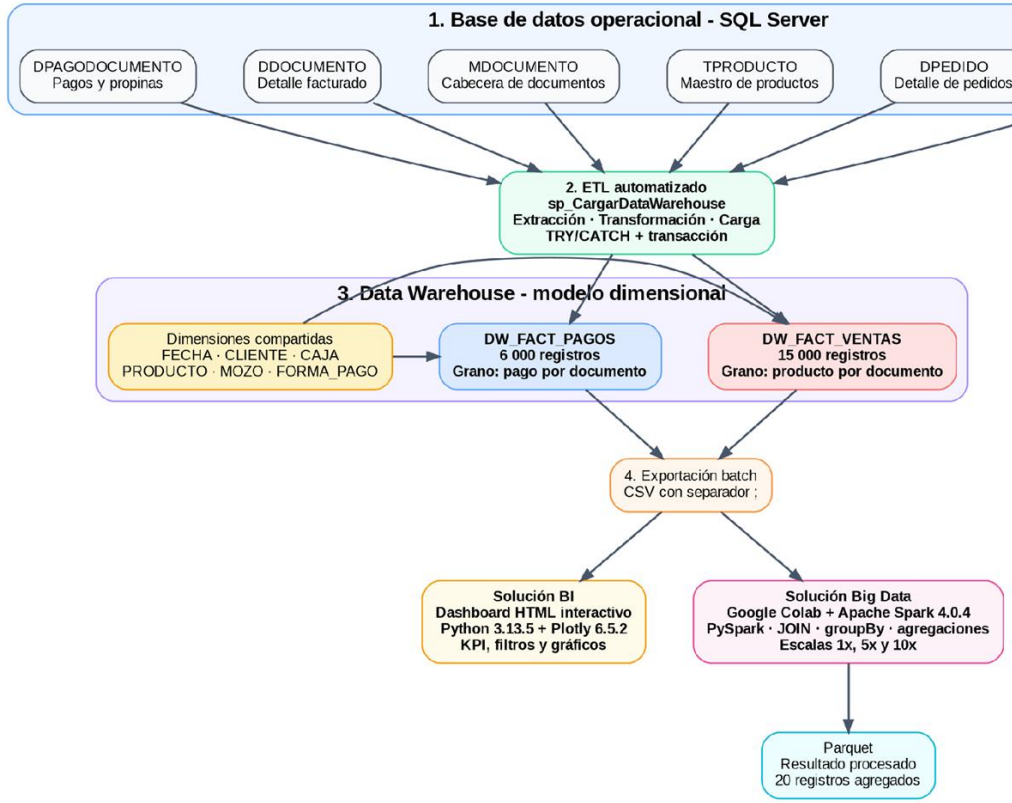
Tecnologías

SQL Server	Operacional + DW
SQL / ETL	Procesamiento por lotes
Power BI	Análisis y visualización
Apache Spark	Procesamiento distribuido
Python / PySpark	Experimentación

Idea clave: primero consolidar una base BI confiable; luego medir si el crecimiento de datos justifica procesamiento distribuido.

Diseño de ingeniería de las soluciones

Arquitectura integrada: SQL Server → ETL → Data Warehouse → BI + Big Data



Modelo dimensional

6 dimensiones + 2 hechos separados para evitar duplicar ventas y pagos.

VENTAS	PAGOS	DW
15 000	6 000	21 315
registros	registros	registros

ETL automatizado

Procedimiento sp_CargarDataWarehouse con TRY/CATCH, transacción y carga ordenada: dimensiones → hechos.

Tiempo promedio ETL: 0.406 s

Calidad y reconciliación

0 nulos críticos en hechos; ventas y pagos reconciliados al 100%.

S/ 365 122 = ventas = pagos

CAPÍTULO IV

Diseño del experimento y validación

Pruebas reproducibles sobre el Data Warehouse: calidad, BI y escalabilidad Big Data

Pregunta experimental

¿La arquitectura permite transformar datos operacionales en información analítica manteniendo integridad al escalar el volumen?

Datos base validados

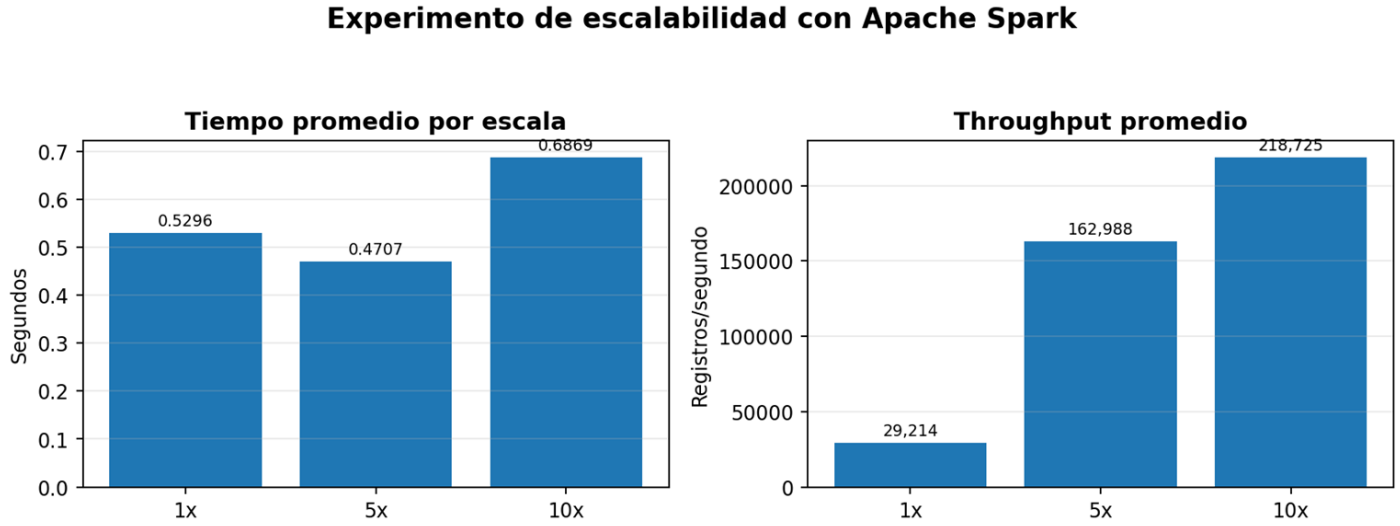
- Base operacional: 41 020 registros
- Data Warehouse: 21 315 registros
- Ventas / pagos: S/ 365 122 reconciliados
- Parquet: 20 registros agregados

Escenarios

1x = 15 000 ventas
5x = 75 000 ventas
10x = 150 000 ventas

Entorno Big Data

Spark 4.0.4 · Python 3.13.16 · Colab: 2 CPU y 12.67 GB RAM



Fuente: ProyectoFinal_BigData_PySpark.ipynb (3 repeticiones por escenario).

Lectura del experimento

Las 9 ejecuciones conservaron registros, unidades y ventas esperadas. El throughput aumentó con la escala; en Colab, los tiempos pequeños reflejan sobrecarga fija y optimización del motor Spark.

CAPÍTULO V

Implementación y control de evidencias

Soluciones BI y Big Data ejecutadas sobre el Data Warehouse, con pruebas verificables y trazabilidad del equipo

Implementación

Solución BI

- Base operacional: 6 tablas, 41 020 registros
- Data Warehouse: 6 dimensiones + 2 hechos (21 315 registros)
- ETL: sp_CargarDataWarehouse con transacción y TRY/CATCH
- Dashboard en Power BI conectado a SQL Server (filtros por fecha y caja)

Solución Big Data

- Spark 4.0.4 en Google Colab (local[*]); ingesta de los CSV del DW
- Normalización de códigos con lpad (ceros a la izquierda)
- Salida en Parquet y experimento de escalabilidad 1x / 5x / 10x
- 9 ejecuciones con validación de registros, unidades y ventas

Pruebas de calidad: 10 de 12 cumplidas

Prueba	Resultado	Estado
ETL automatizado (RNF-03)	0,406 s promedio (< 1 s)	✓ Cumple
Nulos críticos (RNF-01)	0 en origen y hechos	✓ Cumple
Ventas = pagos (RNF-02)	S/ 365 122; dif. S/ 0,00	✓ Cumple
Dashboard Power BI (RF-05, RF-06)	4 de 6 KPI; filtros fecha y caja	Parcial
Spark 1x / 5x / 10x (RF-09)	9 de 9 validaciones	✓ Cumple
Escala 10x < 2 s (RNF-04)	0,6869 s	✓ Cumple

Pendiente: KPI faltantes en el dashboard y archivos finales en el repositorio (RNF-05).

Control de evidencias y equipo

Repositorio: github.com/LilKxng09/basav-bi-bigdata-restaurante

Jhon: cap. I–II · Erick: base de datos y modelo, cap. III–IV · David: BI en Power BI, cap. V–VI · Brenda: ETL, cap. VII–VIII. Uso de IA declarado.

CAPÍTULO VI

Ejecución y resultados

Validación del dataset y desempeño de Spark al escalar de 15 000 a 150 000 registros

Validación de datos

- Original → procesado: 41 020 → 21 315 registros
- 0 nulos críticos en origen y hechos
- Ventas = pagos = S/ 365 122 (diferencia S/ 0,00)
- SQL Server vs. Spark: mismos conteos, KPI y rankings
- Parquet: 20 registros, 17 142 unidades, S/ 365 122
- ETL completo: 0,406 s promedio (381, 463 y 374 ms)
- Escalas sintéticas: 9 de 9 validaciones verdaderas

Resultados Spark por escala (3 repeticiones)

Escala	Filas	Prom. (s)	Mín–máx (s)	Desv. (s)	Reg/s
1x	15 000	0,5296	0,4102–0,6236	0,1090	29 214
5x	75 000	0,4707	0,3853–0,5577	0,0862	162 988
10x	150 000	0,6869	0,6504–0,7155	0,0333	218 725

Entorno: Google Colab (2 CPU, 12,67 GB RAM), Spark 4.0.4 en modo local. Ejecución del 10/10/2026.

Lectura del resultado

Con 10 veces más datos el tiempo subió solo 1,3 veces (0,53 s → 0,69 s) y el throughput se multiplicó por 7,5. La escala 10x cumple el umbral de 2 s.

CAPÍTULO VI

Qué demuestran los resultados y qué no permiten afirmar

Hallazgos

• ETL en 0,406 s y reconciliación del 100 %

• Spark: 10x datos → 1,3x tiempo; throughput ×7,5

• 5x más rápido que 1x: sobrecarga fija de la JVM y variabilidad de Colab (los rangos se solapan)

• Único defecto de calidad: ceros perdidos por inferSchema, corregido con lpad

Límites

• Escalas 5x y 10x por replicación: solo sirven para rendimiento

• Spark local en un nodo; 3 repeticiones por escala

• No se midió SQL Server: no se afirma que Spark sea más rápido

• 150 000 filas sigue siendo un volumen pequeño

Verificación

Producto	Estado
Solución BI	Parcial
Solución Big Data	✓ Cumple (local)
Integración y repositorio	Parcial

Conclusión

La arquitectura conserva la integridad y escala bien en el entorno probado; falta evidencia en clúster.

Responsabilidad ética y profesional

Tratamiento de datos

Datos de clientes, mozos y pagos requieren autorización, acceso restringido y política de retención

Impactos

Mejora decisiones del negocio, pero hay riesgo de generalizar a partir de una muestra pequeña

Compromiso del equipo

Transparencia sobre las limitaciones del volumen de prueba y reproducibilidad del proceso ETL

Conclusiones técnicas y recomendaciones

Conclusiones clave

•Modelo estrella con 6 dimensiones y 2 tablas de hechos, validado con 9 registros de ventas y 6 de pagos

•Separar ventas y pagos evitó duplicar registros por múltiples formas de pago

•ETL con DISTINCT y carga ordenada garantizó integridad referencial

•Dashboard en Power BI responde a los objetivos de negocio planteados

Recomendaciones

•Automatizar el proceso ETL con un job programado

•Añadir controles de calidad automatizados antes de la carga

•Incluir segmentadores interactivos en el dashboard

•Evaluar dimensiones de cambio lento (SCD) a futuro